CS2102 Database Systems

Ryan Joo

AY25/26 Sem 2

The aim of this course is to introduce the fundamental concepts and techniques necessary for the understanding and practice of design and implementation of database applications and of the management of data with relational database management systems. The course covers practical and theoretical aspects of design with entity-relationship model, theory of functional dependencies and normalisation by decomposition in second, third and Boyce-Codd normal forms. The course covers practical and theoretical aspects of programming with SQL data definition and manipulation sublanguages, relational tuple calculus, relational domain calculus and relational algebra.

Prerequisite

If undertaking an Undergraduate Degree THEN(must have completed 1 of CS1231 / CS1231S / MA1100 / MA1100T at a grade of at least D AND must have completed 1 of CS2030 / CS2030DE / CS2030S / CS2040 / CS2040C / CS2040DE / CS2040HS / CS2040S / YSC2229 at a grade of at least D)

Preclusion

If undertaking an Undergraduate Degree THEN(must not have completed 1 of CS2102S / IT2002 at a grade of at least D)

Tips to write a query:

(1) Identify tables and number of tables needed (e.g. multiple copies) - need/how to join?

(2) Identify attributes needed

(3) Identify columns needed

(4) Identify conditions for WHERE clause

Contents

Introduction

A **database** is a collection of data, typically describing the activities of one or more related organizations. For example, a university database might contain information about the following:

- **Entities** such as students, faculty, courses, and classrooms.
- **Relationships** between entities, such as students' enrollment in courses, faculty teaching courses, and the use of rooms for courses.

A **database management system** (DBMS) is software designed to assist in maintaining and utilizing large collections of data.

1 Creating and Populating Tables with Constraints

In terms of notation, elements in `<...>` is a production rule; elements in `[...]` is optional.

1.1 Logical design

The main construct of the relational model is a *relation*.

Definition 1.1. A **relation** consists of a **relation schema** and a **relation instance**.

- The **relation schema** specifies the structure of the relation: its name, its attributes, and the domain of each attribute.
- The **relation instance** is a set of **tuples** (records) that conform to the schema. Each tuple assigns a value from the corresponding domain to every attribute.

  A relation instance can be thought of as a table in which each tuple is a row, and all rows have the same number of fields.

  (The term *relation instance* is often abbreviated to just *relation*, when there is no confusion with other aspects of a relation such as its schema.)

Formally, let

$$R(A_1 : \texttt{D1}, \ldots, A_n : \texttt{Dn})$$

be a relation schema. For each attribute A_i, let Dom_i be the set of values defined by domain `Di`. An instance of R is a set of tuples of the form

$$(A_1 : d_1, \ldots, A_n : d_n) \quad \text{where} \quad d_i \in Dom_i.$$

We write $R.A_i$ to denote attribute A_i of relation R.

The **degree** (**arity**) of a relation is the number of attributes. The **cardinality** of a relation instance is the number of tuples in it.

A **relational database** is a collection of relations with distinct relation names.

The **relational database schema** is the collection of schemas for the relations in the database.

An **instance** of a relational database is a collection of relation instances, one per relation schema in the database schema.

The `CREATE TABLE` statement is used to define a new table.

```
01 | CREATE TABLE [IF NOT EXISTS] <table name> (
02 |   <attribute name>  <attribute type>  [<column constraint>],
03 |     :,
04 |   <attribute name>  <attribute type>  [<column constraint>],
05 |   [<table constraint>],
06 |     :,
07 |   [<table constraint>]
08 | );
```

Basic data types:

- `BOOLEAN` : logical boolean (true/false)
- `INTEGER` (`INT`): signed 4-bytes integer
- `FLOAT8` : double precision 8-bytes floating point
- `NUMERIC[(p,s)]` : exact numeric of selectable precision
- `CHAR(n)` : fixed-length character string

- `VARCHAR(n)` : variable-length character string
- `TEXT` : variable-length character string
- `DATE` : calendar date (year, month, day)
- `TIMESTAMP` : date and time

1.2 Constraints check

Constraints

Definition 1.2. An **integrity constraint** is a logical condition that every legal database instance must satisfy. A DBMS enforces integrity constraints by rejecting operations that would violate them.

Definition 1.3. A database is in a **consistent state** if all structural and integrity constraints are satisfied.

Definition 1.4. A **transaction** is a sequence of operations executed as a single logical unit of work.

A transaction satisfies the ACID properties:

- **Atomicity**: Either all operations take effect (commit), or none take effect (rollback).
- **Consistency**: A transaction moves the database from one valid state to another while following all rules and constraint.
- **Isolation**: Transactions run independently of each other, even when executed at the same time.
- **Durability**: Once a transaction is committed, its changes remain permanent.

There are two types of constraints in a transaction:

- **Immediate constraint**: checked after each statement.
- **Deferrable constraint**: checked at the end of transaction.

When a `UNIQUE` , `PRIMARY KEY` , or `FOREIGN KEY` constraint is created, it is assigned one of the following enforcement characteristics:

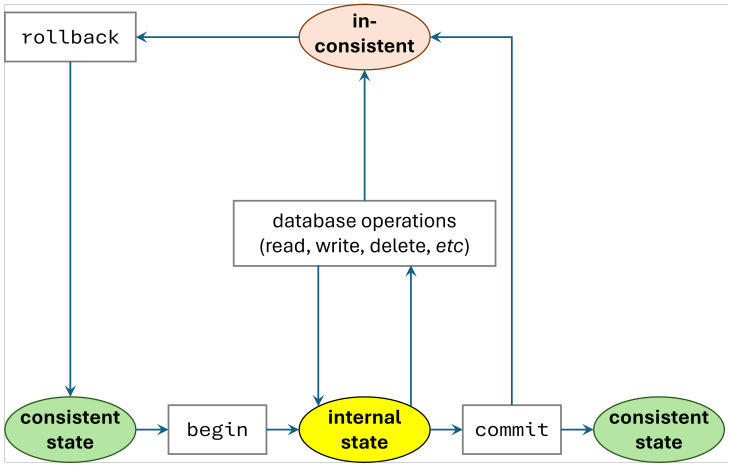
- `DEFERRABLE INITIALLY IMMEDIATE`
- `DEFERRABLE INITIALLY DEFERRED`
- `NOT DEFERRABLE`

These specifications determine **when** the DBMS checks whether the constraint is satisfied.

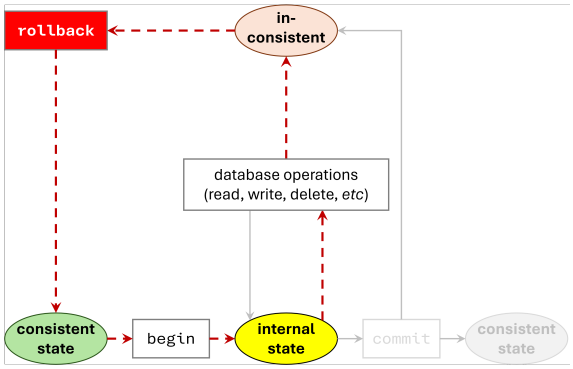
- `NOT DEFERRABLE` The constraint is always checked immediately after each statement (`INSERT` , `UPDATE` , `DELETE`). This is the default behaviour.
- `DEFERRABLE INITIALLY IMMEDIATE` The constraint is checked immediately by default, but its checking time can be postponed to the end of the transaction if explicitly deferred.
- `DEFERRABLE INITIALLY DEFERRED` The constraint is checked only at transaction commit by default, though it may be switched to immediate checking if required.

An explicit transaction starts from `BEGIN` , and ends with either commit or rollback. If no explicit `BEGIN` , each statement is a single transaction.

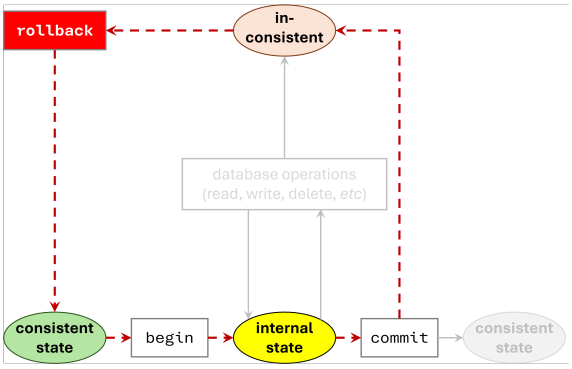
```
01 | BEGIN;
02 | UPDATE accounts SET balance = balance - 100.00
03 |   WHERE name = 'Alice';
04 | -- etc etc
05 | COMMIT;
```



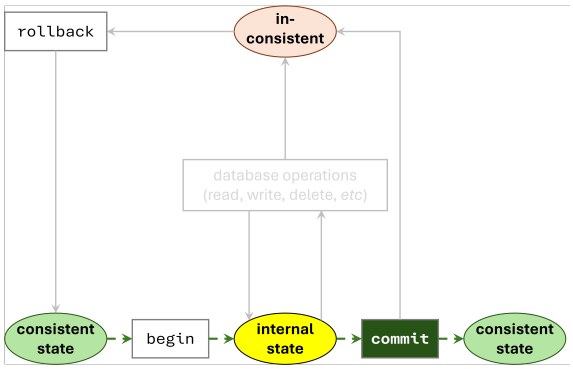
If immediate constraints are inconsistent, we **rollback** to the last consistent state. All changes reverted.



If deferrable constraints are inconsistent, we **rollback** to the last consistent state. Only check on commit.



If there are no violations to the integrity constraints, changes are **committed**. All changes permanent.



There are five kinds of integrity constraints:

(1) `PRIMARY KEY` : Unique combination of values and cannot be NULL

(2) `NOT NULL` : Value cannot be NULL

(3) `UNIQUE` : Unique combination of values

(4) `FOREIGN KEY` : Value exist in referenced table or value contains NULL

(5) `CHECK` : Arbitrary Boolean expression check

Integrity constraints can be specified for columns or tables:

- **Column constraint**: applies to a single column, specified at column definition
- **Table constraint**: applies to one/more columns, specified after column definition (need to specify column name within parentheses)

Reset

We can delete tuples using the `DELETE` command. Note that `DELETE` deletes the content of the table but NOT its definition.

```
01 | DELETE FROM <table_name>
02 | WHERE <condition>;
```

By the **principle of acceptance**, we perform the operation if the condition evaluates to True.
`DROP` deletes the content and the definition of the table.

```
01 | DROP TABLE [IF EXISTS] <table_name> [CASCADE]
```

Query a table:

```
01 | SELECT * FROM customers;
```

Tuples are inserted using the `INSERT` command.

```
01 | INSERT
02 | INTO <table_name> [(<attribute>, ...)]
03 | VALUES (<value>, ...)
04 |   [, (<value>, ...), ...];
```

Remark. We can optionally omit the list of column names in the `INTO` clause and list the values in the appropriate order, but it is good style to be explicit about column names.

Remark. If values are missing, it will be replaced with either default values (if specified) or `NULL` values. You cannot "skip" attributes (e.g., missing for column 2 but specifying column 3).

We can modify the column values in an existing row using the `UPDATE` command.

```
01 | UPDATE <table_name>
02 | SET <column_1> = <expr_1>,
03 |   <column_2> = <expr_2>,
04 |     :
05 |   <column_n> = <expr_n>
06 | [ WHERE <condition> ];
```

Primary Key

Definition 1.5. A **candidate key** is a minimal set of attributes whose values uniquely identify a tuple.

Definition 1.6. A **primary key** is *one* candidate key chosen by the database designer.

(1) Uniqueness: no two tuples share the same key values.

(2) Non-nullability: every key attribute must be non- `NULL` (since `NULL` cannot identify a tuple).

Column constraint:

```
01 | CREATE TABLE customers (
02 |   first_name  VARCHAR(64),
03 |   last_name   VARCHAR(64),
04 |   email       VARCHAR(64),
05 |   dob         DATE,
06 |   since       DATE,
07 |   customerid  VARCHAR(16)
       PRIMARY KEY,
08 |   country     VARCHAR(16)
09 | );
```

Table constraint:

```
01 | CREATE TABLE customers (
02 |   first_name  VARCHAR(64),
03 |   last_name   VARCHAR(64),
04 |   email       VARCHAR(64),
05 |   dob         DATE,
06 |   since       DATE,
07 |   customerid  VARCHAR(16),
08 |   country     VARCHAR(16),
09 |   PRIMARY KEY (customerid)
10 | );
```

```
01 | CREATE TABLE games (
02 |   name        VARCHAR(32),
03 |   version     CHAR(3),
04 |   price       NUMERIC,
05 |   PRIMARY KEY (name, version)
06 | );
```

In the schema notation, we underline the primary key columns:

customers(first_name, last_name, email, dob, since, customerid, country)

Not NULL

No value of the corresponding field in any record of the table can be set to `NULL`.
Column constraint:

```
01 | CREATE TABLE games (
02 |   name        VARCHAR(32),
03 |   version     CHAR(3),
04 |   price       NUMERIC NOT NULL,
05 |   PRIMARY KEY (name, version)
06 | );
```

Implicit NULL: If we specify fewer values than there are attributes, the missing values are implicitly `NULL`.

```
01 | INSERT INTO games (name, version)
02 | VALUES ('Aerified2', '1.0');
```

Explicit NULL: Explicitly provide the value `NULL`.

```
01 | INSERT INTO games
02 | VALUES ('Aerified2', '1.0', NULL);
```

If the value is not provided (i.e., missing value), then the **default value** is used.

```
01 | CREATE TABLE games (
02 |   name         VARCHAR(32),
03 |   version      CHAR(3),
04 |   price        NUMERIC NOT NULL DEFAULT 1.00,
05 |   PRIMARY KEY (name, version)
06 | );
```

Unique

A `UNIQUE` constraint on a column or a combination of columns guarantees the table cannot contain two records with the same value in the corresponding column or combination of columns.

Column or table constraint:

```
01 | CREATE TABLE customers (
02 |   first_name   VARCHAR(64),
03 |   last_name    VARCHAR(64),
04 |   email        VARCHAR(64) UNIQUE,
05 |   dob          DATE,
06 |   since        DATE,
07 |   customerid   VARCHAR(16),
08 |   country      VARCHAR(16),
09 |   UNIQUE (first_name, last_name)
10 | );
```

`PRIMARY KEY` is equivalent to `UNIQUE` and `NOT NULL`. We call such alternative primary key a **candidate key** (i.e., a candidate for primary key).

Foreign Key

Sometimes the information stored in a relation is linked to the information stored in another relation. If one of the relations is modified, the other must be checked, and perhaps modified, to keep the data consistent. A **foreign key constraint** is used to make such checks.

On every insertion/updated on the **referencing table**, the foreign key constraint checks for the **existence** of the value in the **referenced table**.

Remark. For portability, the referenced column is required to be the primary key.

Referenced table:

```
CREATE TABLE customers (
  first_name   VARCHAR(64),
  last_name    VARCHAR(64),
  email        VARCHAR(64),
  dob          DATE,
  since        DATE,
  customerid   VARCHAR(16)
    PRIMARY KEY,
  country      VARCHAR(16)
);
```

Referencing table:

```
CREATE TABLE downloads (
  customerid   VARCHAR(16)
    REFERENCES customers (
    customerid),
  name         VARCHAR(32),
  version      CHAR(3),
  FOREIGN KEY (name, version)
    REFERENCES games (name,
    version)
);
```

```
CREATE TABLE games (
  name         VARCHAR(32),
  version      CHAR(3),
  price        NUMERIC,
  PRIMARY KEY (name, version)
);
```

When a row is inserted into the referencing table `downloads`, the foreign key constraint checks that `customerid` must exist in `customerid` attribute of the referenced table `customer` table (or be `NULL`).

The following deletion of French customers is prevented, because some of them have downloaded some games. If the deletion is allowed, then some tuples in `downloads` would contain foreign key values that do not match any primary key in `customers`; this violates the foreign key constraint. Hence, the operation is prevented.

```
DELETE FROM customers
WHERE country = 'France';
```

For each row in the referencing relation R_1,

- the combination of values of the attributes exist in the referenced relation R_2, or
- one of the values in the given attributes is `NULL` in the referencing relation R_1

Check

A **check constraint** enforces any other condition that can be expressed as a Boolean expression. The constraint is checked on each insert/update.

A `CHECK` constraint is satisfied if the Boolean expression evaluates to `True` or `NULL`. Therefore, unless combined with `NOT NULL`, a `CHECK` constraint does not prevent `NULL` values.

```
CREATE TABLE games (
  name         VARCHAR(32),
  version      CHAR(3),
  price        NUMERIC NOT NULL
    CHECK (price > 0),
  PRIMARY KEY (name, version)
);
```

The operation below is prevented, because discounting all the prices by $5.5 creates negative prices. Thus, this operation is aborted and rolled back automatically.

```
UPDATE games
SET price = price - 5.5;
```

1.3 Updates

As a motivation, consider the following definition and instances:

```
01 | CREATE TABLE downloads (
02 |   customerid  VARCHAR(16)
03 |     REFERENCES customers (customerid),
04 |   name        VARCHAR(32),
05 |   version     CHAR(3),
06 |   FOREIGN KEY (name, version)
07 |     REFERENCES games (name, version)
08 | );
```

downloads

name	version	customerid
Skype	1.0	tom1999
Comfort	1.1	john88
Skype	2.0	tom1999

customers

customerid	email	...
tom1999	tlee@gmail.com	...
john88	al@hotmail.com	...
walnuts	dcs@nus.edu.sg	...

We want to delete the customer `tom1999`; ideally, whenever we delete a customer, we also delete all the games he downloaded. However, by default, this deletion is prevented due to foreign key constraint.

The keyword `CASCADE` **propagates** changes in the referenced table to referencing tables.

- `ON DELETE CASCADE`: Deleting a referenced tuple deletes all referencing tuples.
- `ON UPDATE CASCADE`: Updating a referenced key updates matching foreign keys.

This preserves referential integrity automatically.

```
01 | CREATE TABLE downloads (
02 |   customerid  VARCHAR(16)
03 |     REFERENCES customers (customerid)
04 |     ON UPDATE CASCADE
05 |     ON DELETE CASCADE,
06 |   name        VARCHAR(32),
07 |   version     CHAR(3),
08 |   FOREIGN KEY (name, version)
09 |     REFERENCES games (name, version)
10 |     ON UPDATE CASCADE
11 |     ON DELETE CASCADE
12 | );
```

2 Simple Queries

2.1 Single table

SQL is a **declarative** language. In a declarative language, we describe the output that we want, instead of how to compute the output.
In contrast, in an **imperative** language (e.g., Python), we describe how to compute the output. This depends on the representation of the table in our program (e.g., a table as a list of lists).

Selection

A simple SQL query includes the following clauses:

- `SELECT` clause to indicates the **output columns**
- `FROM` clause that indicates the **table(s) to be queried**
- an optional `WHERE` clause that indicates **condition(s)** on the output

```
01 | SELECT first_name, last_name
02 | FROM customers
03 | WHERE country = 'Singapore';
```

How to read:

> From the given relation customers, find the rows where the country is equal to Singapore. Then, select only the first name and last name.

The wildcard `*` indicates that the output should contain all the column names, in the order of the `CREATE TABLE` declaration.

```
01 | SELECT * FROM customers;
```

`DISTINCT` eliminates eventual duplicates in the result. The output contains only distinct rows.

```
01 | SELECT DISTINCT name, version -- the pair (name, version) is distinct
02 | FROM downloads
03 | WHERE name = 'Aerified';
```

Ordering

`ORDER BY` sorts the output using the given columns. By default, the order is ascending `ASC`; descending is `DESC`.

```
01 | SELECT *
02 | FROM downloads
03 | WHERE name = 'Aerified'
04 | ORDER BY version ASC;
```

Remark. We first do the ordering, then remove attributes. Thus, we can order by attributes not mentioned in the `SELECT` clause.

We can specify multiple columns in `ORDER BY` clause. It uses **stable sorting**:

(1) Sort based on first column.

(2) For any ties, sort based on second columns.

(3) ... until no more columns in `ORDER BY` clause.

```
SELECT *
FROM downloads
WHERE name = 'Aerified'
ORDER BY version ASC,
         customerid DESC;
```

We can combine `DISTINCT` with `ORDER BY`. However, this is only valid if every attribute in the `SELECT` clause also appears in the `ORDER BY` clause.

```
-- Valid: the selected attribute appears in ORDER BY
SELECT DISTINCT customerid
FROM downloads
WHERE name = 'Aerified'
ORDER BY customerid DESC;
```

If an attribute in `SELECT` does not appear in `ORDER BY`, the ordering is ambiguous, and the query may produce multiple valid outputs. In such cases, SQL returns an **error**.

```
-- Invalid: 'name' is selected but not in ORDER BY
SELECT DISTINCT name
FROM games
WHERE price > 1
ORDER BY version ASC,
         price ASC;
```

Operation

We **filter rows** using Boolean conditions in `WHERE` clause. Complex conditions can be constructed using logical operators (`AND`, `OR`, `NOT`) as well as relational operators (`=`, `<>`, `>`, `<`, `>=`, `<=`, `BETWEEN .. AND`, `LIKE`, etc).

Renaming uses `AS` keyword.

```
SELECT DISTINCT price * 1.09 AS gst   -- add GST of 9%
FROM games
ORDER BY gst ASC;
```

Concatenation uses `||` symbol.

```
SELECT name || ' ' || version AS game,
       ROUND(price * 1.09, 2) AS price  -- add GST of 9%
FROM games
WHERE price * 0.09 >= 0.3;
```

If-else statements use the `CASE-WHEN` syntax.

```
SELECT name || ' ' || version AS game,
  CASE
    WHEN price >= 5 THEN price * 2
    WHEN price * 0.09 >= 0.3 THEN ROUND(price * 1.09, 2)
    ELSE price
  END AS price
FROM games;
```

The `LIKE` expression returns true if the `string` **matches** the supplied `pattern`.

```
01 | string LIKE pattern [ESCAPE escape-character]
```

If `pattern` does not contain percent signs or underscores, then the pattern only represents the string itself; in that case `LIKE` acts like the equals operator. An underscore `_` in `pattern` stands for (matches) any single character; a percent sign % matches any sequence of zero or more characters.

```
01 | 'abc' LIKE 'abc'    true
02 | 'abc' LIKE 'a%'     true
03 | 'abc' LIKE '_b_'    true
04 | 'abc' LIKE 'c'      false
```

2.2 Three-valued logic

Definition 2.1. Two logical formulas are **equivalent** if they have the same truth table.

In two-valued logic involving True and False, recall:

- Commutativity of `AND`: `P AND Q ≡ Q AND P`
- Associativity of `AND`: `(P AND Q) AND R ≡ P AND (Q AND R)`
- Commutativity of `OR`: `P OR Q ≡ Q OR P`
- Associativity of `OR`: `(P OR Q) OR R ≡ P OR (Q OR R)`
- De Morgan's Law: `NOT (P AND Q) ≡ (NOT P) OR (NOT Q)`, `NOT (P OR Q) ≡ (NOT P) AND (NOT Q)`
- Implication equivalence: `P -> Q ≡ (NOT P) OR Q`

Three-valued logic includes `NULL`. Logically, we treat `NULL` as both true and false. If the result can be both true and false, we treat the result as `NULL`.

P	Q	P AND Q	P OR Q	NOT P
T	T	T	T	F
T	F	F	T	F
T	N	N	T	F
F	T	F	T	T
F	F	F	F	T
F	N	F	N	T
N	T	N	T	N
N	F	F	N	N
N	N	N	N	N

Useful properties:

- `AND` is commutative and associative
- `OR` is commutative and associative
- De Morgan's Law is still applicable.
- Implication equivalence is still applicable.

Theorem 2.2 (Principle of acceptance)**.** *We accept if the evaluation of the condition is True. Otherwise, it is rejected. In other words, false / `NULL` will be rejected.*

Used in the evaluation of `WHERE` clause to determine if a row is in the output.

Theorem 2.3 (Principle of rejection)**.** *We reject if the evaluation of the condition is False. Otherwise, it is accepted. In other words, true / `NULL` will be accepted.*

Used in the evaluation of constraints check (e.g., in `CREATE TABLE`).

Definition 2.4. Every domain has the additional `NULL` value. In SQL, it generally has the meaning of "unknown".

Whenever the answer is "**I don't know**", we use `NULL` .
Any relational operation with `NULL` produces `NULL` values.
Any arithmetic operation with `NULL` produces `NULL` values.

- `something = NULL` is unknown (even if `NULL = NULL`).
- `something <> NULL` is unknown (even if `NULL <> NULL`).
- `something > NULL` is unknown.
- `something < NULL` is unknown.
- `10 + NULL` is `NULL` .
- `0 * NULL` is `NULL` (for some unknown reason).

There are several operations to deal with `NULL` :

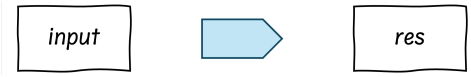
- `x IS NULL`/`x IS NOT NULL` checks if the `x` is `NULL` or not.
- `COALESCE(x1, x2, x3, ...)` returns the first non- `NULL` value.
- `NULLIF(x1, x2)` returns `NULL` if `x1 = x2` , else returns `x1` .

3 Aggregate and Join Queries

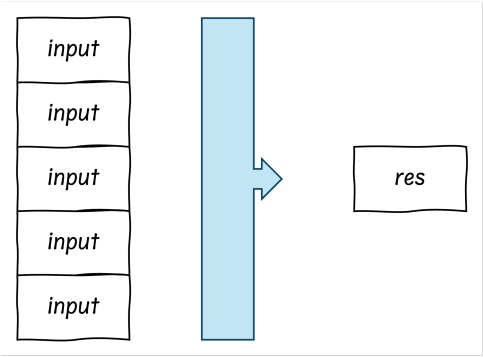
3.1 Aggregation

Functions

Definition 3.1. A **simple function** is a mapping from a single value to a single value.



Definition 3.2. An **aggregation function** is a mapping from a set of values to a single value.



The following are useful aggregation functions:

- `COUNT` : number of rows
- `SUM` : sum of rows
- `AVG` : average of rows (i.e., sum / count)
- `STDDEV` : standard deviation
- `MAX` / `MIN` : largest/smallest row in rows

By default, aggregation function use all rows, including rows with `NULL` . If column is specified, only non- `NULL` rows in the column are used.

```
01 |  -- Counts all rows
02 |  SELECT COUNT(*)
03 |  FROM customers c;
04 |
05 |  -- Counts non-NULL rows of the specified column
06 |  SELECT COUNT(c.customerid)
07 |  FROM customers c;
```

We can wrap `COUNT` around `DISTINCT` to count the number of distinct elements:

```
01 |  SELECT COUNT(DISTINCT c.country)
02 |  FROM customers c;
```

We use the arithmetic `TRUNC()` to display two decimal places for average and standard deviation.

- `TRUNC(v,p)` : **removes** any digits from `v` beyond the specified decimal places `p` .

- `ROUND(v,p)` : **round** the value `v` to the specified decimal places `p` .
- `CEIL(v)` : smallest integer not less than `v` .
- `FLOOR(v)` : largest integer not greater than `v` .

Note that the `WHERE` clause is evaluated before aggregation.

Grouping

The `GROUP BY` clause creates **logical groups** of records that have the same values for the specified fields. Then the aggregation functions are calculated for each logical group.

```
01 | SELECT c.country, COUNT(*)
02 | FROM customers c
03 | GROUP BY c.country;
```

Every expression in the `SELECT` clause must be

- a column used in the `GROUP BY` clause, or
- an aggregate function (used to calculate values within each group).

```
01 | SELECT c.customerid, COUNT(*)  -- c.customerid is an error
02 | FROM customers c
03 | GROUP BY c.country;
```

However, PostgreSQL allows `SELECT` clause to use any attribute from the table, if the **identifying attributes** are in `GROUP BY` .

```
01 | SELECT c.first_name, c.last_name, COUNT(*)
02 | FROM customers c
03 | GROUP BY c.customerid; -- allowed since c.customerid is primary key
```

Renamed columns can be used in a `GROUP BY` clause, provided the renamed expression does not involve an aggregate function.

```
01 | -- total number of downloads for each country and registration year
02 | SELECT c.country, EXTRACT(YEAR FROM c.since) AS regyear, COUNT(*) AS total
03 | FROM customers c, downloads d
04 | WHERE c.customerid = d.customerid
05 | GROUP BY c.country, regyear
06 | ORDER BY regyear ASC, c.country ASC;
```

The order of columns in `GROUP BY` clause does not change the meaning of the query. The logical groups remain the same.

To select countries with more than 100 customers, we cannot use aggregate functions in `WHERE` , since `WHERE` is applied *before* grouping:

```
01 | SELECT c.country
02 | FROM customers c
03 | WHERE COUNT(*) >= 100 -- error
04 | GROUP BY c.country;
```

SQL provides the `HAVING` clause, which is evaluated after `GROUP BY` . It allows the use of aggregate functions and may refer to columns in the `GROUP BY` clause:

```
01 | SELECT c.country
02 | FROM customers c
03 | GROUP BY c.country
04 | HAVING COUNT(*) >= 100;
```

3.2 Joins

Cross Join

The **Cartesian product** of tables is the table containing all the columns and all possible combinations of the rows of the three tables.
The comma in the `FROM` clause is a convenient shorthand for the keyword `CROSS JOIN`.

```
01 | SELECT *
02 | FROM customers,
03 |      downloads,
04 |      games;
```

```
01 | SELECT *
02 | FROM customers
03 |   CROSS JOIN downloads
04 |   CROSS JOIN games;
```

Note the following properties:

- The number of columns is the sum of number of columns of each table.
- The number of rows is the product of the number of rows of each table.

However, such a joining of tables is not meaningful, as it includes many irrelevant rows. To be more precise, we add the **join condition** that

foreign key columns are equal to the corresponding primary key columns.

The condition below meaningfully corresponds to the "customers who download a game".

```
01 | SELECT *
02 | FROM customers c, downloads d, games g
03 | WHERE d.customerid = c.customerid
04 |   AND d.name = g.name
05 |   AND d.version = g.version;
```

Remark. It is required from now on in this course to define an **alias** for each table in the `FROM` clause, and use the **dot notation** to avoid ambiguity.

Underneath the join condition, we can proceed to add more filter conditions in the `WHERE` clause.

```
01 | SELECT c.email, g.version
02 | FROM customers c, downloads d, games g -- three tables
03 | WHERE c.customerid = d.customerid
04 |   AND g.name = d.name
05 |   AND g.version = d.version -- join condition
06 |   AND c.country = 'Indonesia'
07 |   AND g.name = 'Fixflex'; -- filter conditions
```

Inner Join

In inner join, the join condition is specified in the attached `ON` clause (which is mandatory).

```
SELECT *
FROM customers c
  INNER JOIN downloads d
    ON d.customerid = c.customerid
  INNER JOIN games g
    ON d.name = g.name
    AND d.version = g.version;
```

Natural Join

`NATURAL JOIN` joins rows that have the same values for columns with the **same name**.

It also prints only one of the two columns (only appear once), in contrast to cross join which may have duplicate columns.

```
SELECT *
FROM customers c
  NATURAL JOIN downloads d
  NATURAL JOIN games g;
```

A **universal relation** is a relation in which columns with the same name have the same meaning. This ensures that `NATURAL JOIN` can be used naturally.

Outer Join

A **dangling row** is a row from the specified table that does not match any rows from the other table according to the join condition.

In an outer join, the **specified table** determines which rows are preserved even when there is no matching row in the other table:

`L_TABLE <outer join> R_TABLE`

- `LEFT OUTER JOIN` : The specified table is `L_TABLE` .

  All rows from `L_TABLE` are retained, including those with no match in `R_TABLE` . Columns from `R_TABLE` for unmatched rows are filled with `NULL` (`NULL` padding).

- `RIGHT OUTER JOIN` : The specified table is `R_TABLE` .

  All rows from `R_TABLE` are retained, and unmatched columns from `L_TABLE` are `NULL` .

- `FULL OUTER JOIN` : Both `L_TABLE` and `R_TABLE` are preserved.

  Rows with no match in the other table receive `NULL` for the missing columns.

```
-- Find the different customers who have not downloaded any games
-- 1. Customers who has downloaded at least one game is based on the
    condition: c.customerid = d.customerid
-- 2. Using LEFT OUTER JOIN, we include customers who have not downloaded
    any games but with NULL padding
-- 3. Keep only the dangling rows by checking for NULL values for attributes
     that should not be NULL
SELECT c.customerid, c.email, d.customerid, d.name, d.version
FROM customers c
  LEFT OUTER JOIN downloads d
    ON c.customerid = d.customerid
WHERE d.customerid IS NULL;
```

3.3 Set operations

Definition 3.3. Two queries are **union-compatible** if they return the same number of columns with the same domain (type) in the same order.

The set operators `UNION`, `INTERSECT`, and `EXCEPT` return the union, intersection, and non-symmetric difference of the results of two queries respectively.

Two queries must be union-compatible to be used with UNION, INTERSECT, or EXCEPT.

Deduplication: Union, intersection, and non-symmetric difference **eliminate duplicates**, unless annotated with the keyword `ALL` (keep duplicates).

4 Entity-Relationship Model

Given a text, steps to draw Entity-Relationship Diagram:

(1) Identify the **entity sets**.

These generally correspond to *nouns* (but not proper nouns).

(2) Identify the **relationship sets** and the entity sets that they associate.

These generally correspond to *verbs*. However, you may need to infer its presence as well as its name.

(3) For each entity set and relationship set, identify its **attributes**.

(4) For each entity set, identify its **keys**.

(5) For each entity set and each relationship set in which it participates, indicate the minimum and maximum **participation constraints**.

(6) Draw the corresponding entity-relationship diagram with the key and participation constraints. Indicate in English the constraints that cannot be captured, if any.

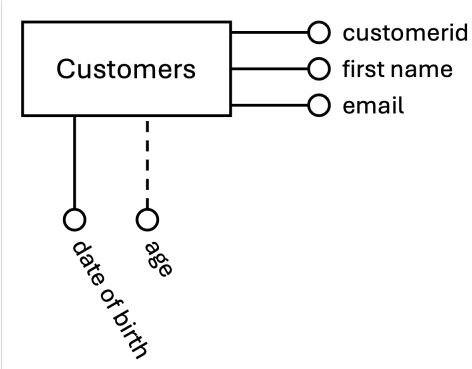
4.1 Entities

Definition 4.1. An **entity** is an object in the real world that is distinguishable from other objects. An **entity set** is a collection of similar entities (of the same type).

Entities typically correspond to **nouns** in the description.
In ERD, an entity is represented as a **named box**.

Definition 4.2. An entity is described using a set of **attributes**. All entities in a given entity set have the same attributes; this is essentially what we mean by *similar*.

For each attribute associated with an entity set, we must identify a **domain** of possible values.
A **derived attribute** is an attribute whose value can be **derived** from other attributes (via computation).
In ERD, attributes are represented by a **circle** with names next to the circle. A derived attribute is connected with **dashed line**.



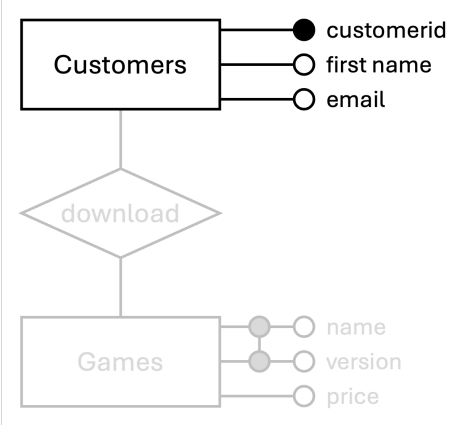
For each entity set, we choose a *key*.

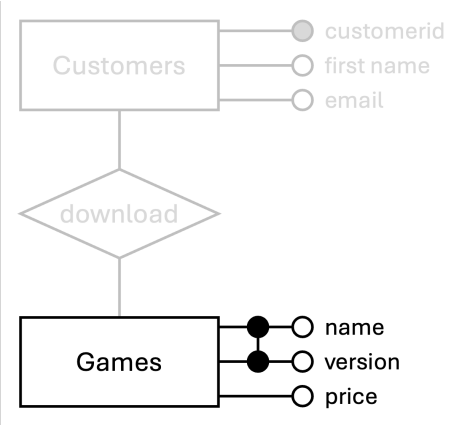
Definition 4.3. A **key** is a minimal set of attributes whose values uniquely identify an entity in the set.

Remark. Minimal is not the same as minimum.

- Minimal means that we cannot remove any element from it while still maintaining the condition. This is like a local minima.
- Minimum means smallest possible. This is like a global minima.

There could be more than one **candidate key**; if so, we designate one of them as the **primary key**.
For now, we will assume that each entity set contains at least one set of attributes that uniquely identifies an entity in the entity set; that is, the set of attributes contains a key.
In ERD, a key is represented as **black circle** or (**connected black circle** for composite key).





4.2 Relationships

Definition 4.4. A **relationship** is an association among two or more entities. A **relationship set** is a set of similar relationships (of the same type).

The relationship is **binary** if it associates 2 entities, and **ternary** if it associates 3 entities.

Remark. As relationship set is a set, there are no duplicates. All the entities in a relationship must be participating; none of the entities can be omitted (e.g., `NULL`), since we cannot identify an entity using `NULL` (unknown).

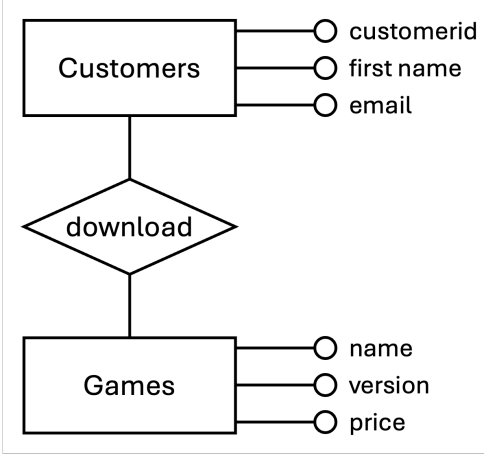
A relationship set can be thought of as a set of n-tuples:

$$\{(e_1, \ldots, e_n) \mid e_1 \in E_1, \ldots, e_n \in E_n\}.$$

Each n-tuple denotes a relationship involving n entities $e_1, \ldots, e_n$, where entity e_i is in entity set E_i.
In the description, relationship typically correspond to **verbs**.

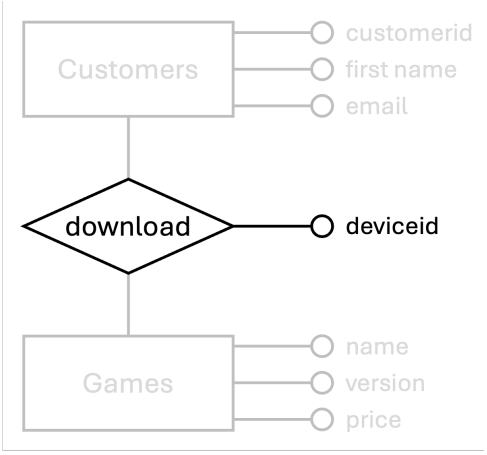
Example. "We record which version of which game each customer has **downloaded**."

In ERD, a relationship is represented as a **named diamond**.



Mathematically, if customer C_1 downloads game G_1, then the pair (C_1, G_1) is an element of the `download` relationship set.

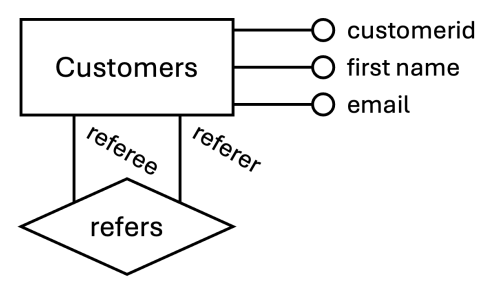
A relationship can also have **descriptive attributes**. Descriptive attributes are used to record information about the relationship, rather than about any one of the participating entities.



A relationship is uniquely identified by the **participating entities**, without reference to the descriptive attributes. In this case, in the `download` relationship set, each relationship is uniquely identified by the combination of customer and game.

Thus, for a given customer-game pair (C_1, G_1), we cannot have more than one associated `deviceid` value, i.e., the pairs (C_1, G_1, D_1) and (C_1, G_1, D_2) cannot exist together. In other words, a customer cannot download the same game multiple times but on a different device.

Relationships can associate entities from the **same entity set**. In this case (and in general), **participation** (or role), in the relationship can be **named**.

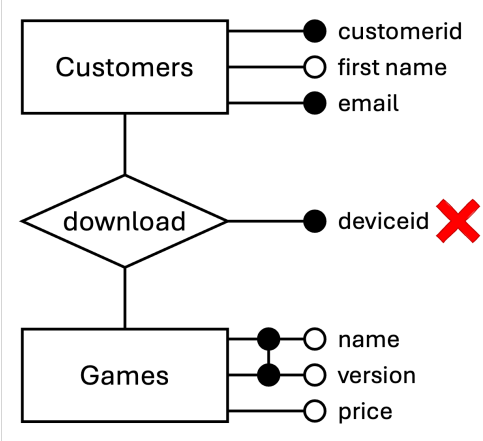


To solve the previous issue of "we still cannot have a customer downloading the same game multiple times but

on a different device.", we can promote an attribute into an entity set. In this case, the pairs (C_1, G_1, D_1) and (C_1, G_1, D_2) are allowed.

Since `downloads` is a relationship set, the same customer cannot download the same game on the same device multiple times.

Since relationship sets are identified by their participating entities and not their attributes, we prohibit relationship set from having black circle(s).



4.3 Weak entities

Thus far, we have assumed that every entity set possesses a key that uniquely identifies each entity. This assumption does not always hold. In some cases, an entity can only be uniquely identified within the context of a relationship with another entity set.

Definition 4.5.

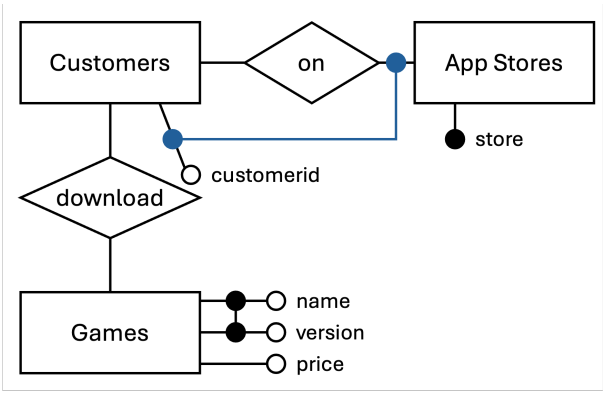
- A **weak entity set** is an entity set that does not have a full key of its own.

  Instead, it contains a **partial key**, which uniquely identifies an entity only relative to a related entity from another set.

- The entity set that provides this identifying context is called the **dominant entity set**.

  The relationship that connects the weak entity set to its dominant entity set is called the **identifying relationship set**.

In ERD, connect the partial identifier to the participation of dominant entity.

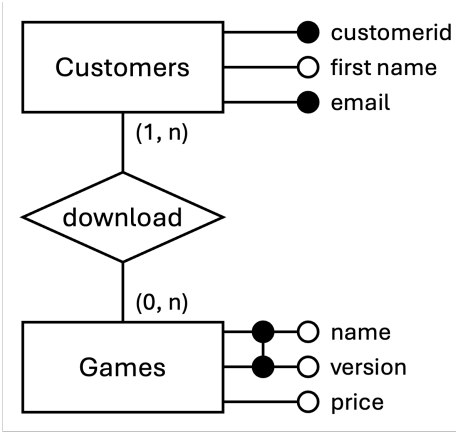


For example, a customer cannot be uniquely identified by `customerid` alone if multiple app stores exist. Different app stores may assign the same identifier to different customers. Hence, a customer is uniquely identified only within a specific app store, and `customerid` functions as a partial key.

combination of customerid and store uniquely identifies

4.4 Participation constraints

Definition 4.6. A **participation constraint** is the number of times an entity can appear in the relationship. It is between an entity set and relationship set and restricted by (minimum, maximum) value.



- (1,_): A customer must download at least one game.
- (_, n): A customer may download many games.
- (0, _): A game need not be downloaded.
- (_, n): A game can be downloaded many times.

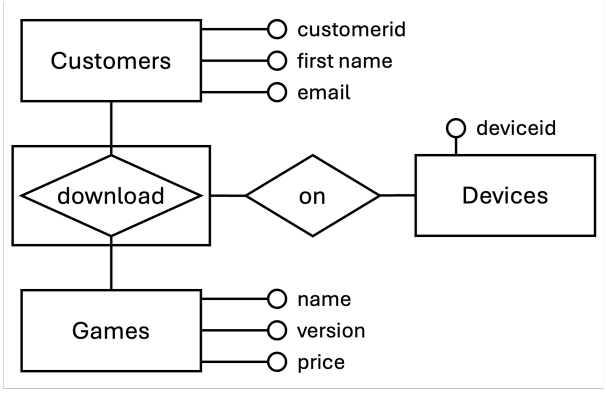
For a binary relationship (i.e., involving two entity sets), we can classify the participation constraints as the following **cardinalities**:

- $(1, \_)$ characterizes a **mandatory** participation.
- $(0, \_)$ characterizes an **optional** participation.
- $(\_, 1)$ may characterizes a **one-to-one** relationship if (_, 1) for all entities involved.
- $(\_, 1)$ may characterizes a **one-to-many** relationship. if (, 1) for one but (, n) or (_, x) for $x > 1$ for other
- $(\_, n)$ characterizes a **many-to-many** relationship. if (_, n) for all

Remark. If participation constraint is omitted, we assume the default participation is $(0, n)$.

4.5 Aggregation

In some situations, we need to model a relationship that itself participates in another relationship. To represent this, ER diagrams allow a relationship set to be treated as a higher-level entity. This abstraction is called **aggregation**. In ERD, wrap the relationship set in a box.



Although it is treated as a higher-level object, an aggregate is fundamentally still a relationship set. Therefore, all standard properties of relationship sets continue to apply:

- A customer cannot download the same game multiple times.

- Each download is uniquely identified by its participating entities.
- All usual constraints governing identifying attributes remain in effect.

4.6 Schema translation

Convert ERD to schema.

Rule 1: Value Sets

Value sets are mapped to **domains**. ER attributes are mapped to attributes of relations with meaningful type.

```
first_name VARCHAR(64) NOT NULL
```

Remark. By default, we expect `NOT NULL` constraint to be used. It is best to avoid having `NULL` values. You need to justify every instance where `NULL` value is needed (i.e., every time you are not adding `NOT NULL`).

Rule 2: Entity Sets

Entity sets are mapped to **relations**. Entity set attributes are mapped to **attributes** of the relation. Candidate keys are mapped to primary keys and `UNIQUE NOT NULL`.

(1) Translate attributes as value sets using Rule 1.

(2) Add `PRIMARY KEY` constraint to identifying attribute.

(3) Add `UNIQUE NOT NULL` constraints to candidate keys.

(4) Add `CREATE TABLE` with the name of the entity set.

Composite keys can be specified using table constraint.

```
CREATE TABLE customers (
  first_name VARCHAR(64) NOT NULL,
  email      VARCHAR(64) UNIQUE NOT NULL,
  customerid VARCHAR(16) PRIMARY KEY,
    -- other attributes omitted
);
```

Rule 3: Relationship Sets

Relationship sets are mapped to **relations**. The attributes of the relation consist of the attributes of the relationship set. The keys are the **keys of the participating entities**.

(1) The attributes of the relationship set (if any).

(2) The primary keys of the participating entity sets.

(3) Reference to the participating entity sets.

Foreign key reference primary key

(4) The keys are the keys of the participating entity sets.

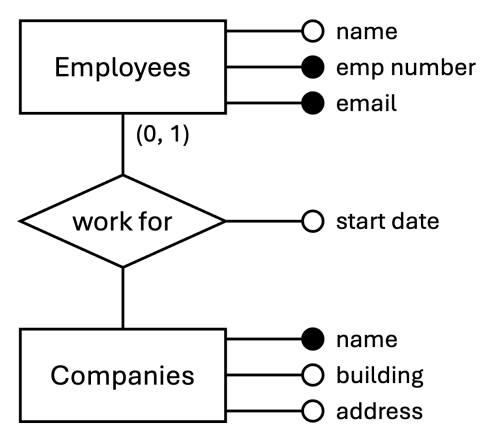
(Recap that a relationship set is identified by the participating entities.)

What About Aggregate? **Aggregate** is simply a **relationship set**. So the rule for relationship set applies. But it can be used as **entity set** in a sense that the keys can be **referenced** by another relationship set.

```
CREATE TABLE downloads(
  customerid VARCHAR(16)
    REFERENCES customers(customerid),
  name       VARCHAR(32),
  version    CHAR(3),
  deviceid   VARCHAR(64) NOT NULL,
  PRIMARY KEY (customerid, name, version),
  FOREIGN KEY (name, version)
    REFERENCES games(name, version)
);
```

Exception 1: One-to-Many

Suppose each employee works for at most one company, while a company may employ many employees. This is a **one-to-many** relationship.



Incorrect design: The composite primary key `(emp_number, name)` allows multiple rows with the same `emp_number` but different `name` values. This permits an employee to work for multiple companies, which violates the one-to-many constraint.

```
CREATE TABLE work_for (
  start_date  DATE NOT NULL,
  emp_number  CHAR(8),
  name        VARCHAR(32),
  PRIMARY KEY (emp_number, name),
  FOREIGN KEY (emp_number)
    REFERENCES employees(emp_number),
  FOREIGN KEY (name)
    REFERENCES companies(name)
);
```

Correct design: Restrict the primary key to the entity set with (0, 1) cardinality.

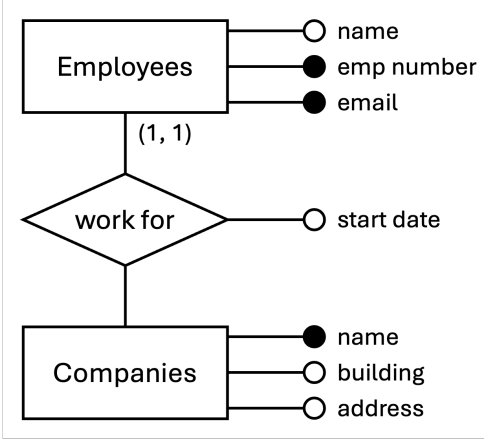
```
CREATE TABLE work_for (
  start_date  DATE NOT NULL,
  emp_number  CHAR(8),
```

```
04 |    name         VARCHAR(32) NOT NULL,
05 |    PRIMARY KEY (emp_number), -- remove name from PK
06 |    FOREIGN KEY (emp_number)
07 |      REFERENCES employees(emp_number),
08 |    FOREIGN KEY (name)
09 |      REFERENCES companies(name)
10 |  );
```

By using `emp_number` as the primary key, each employee can appear at most once in the table. This enforces the constraint that an employee works for no more than one company.

Exception 2: (1, 1)

Suppose every employee must work for exactly one company. This means the participation of employees in the relationship is (1, 1): each employee is associated with one and only one company.



Incorrect design: The employee and relationship are stored in separate tables. This makes it possible to insert an employee into `employees` without inserting a corresponding row into `work_for`, violating minimum participation.

```
01 |  CREATE TABLE work_for (
02 |    start_date   DATE NOT NULL,
03 |    emp_number   CHAR(8),
04 |    name         VARCHAR(32) NOT NULL,
05 |    PRIMARY KEY (emp_number),
06 |    FOREIGN KEY (emp_number) REFERENCES employees(emp_number),
07 |    FOREIGN KEY (name) REFERENCES companies(name)
08 |  );
```

Correct design: Merge `employee` and `work_for`.

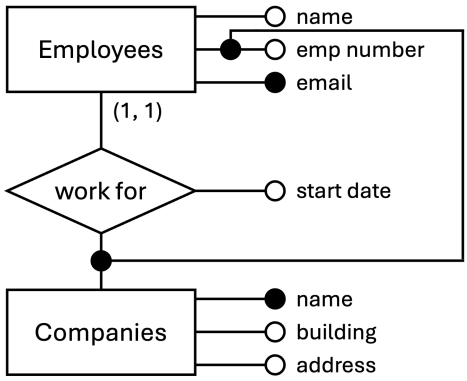
```
01 |  CREATE TABLE employee_work_for (
02 |    start_date   DATE NOT NULL,
03 |    emp_name     VARCHAR(64) NOT NULL,
04 |    emp_number   CHAR(8),
05 |    email        VARCHAR(64) NOT NULL UNIQUE,  -- candidate key
06 |    name         VARCHAR(32) NOT NULL,
07 |    PRIMARY KEY (emp_number),
08 |    FOREIGN KEY (name) REFERENCES companies(name)
09 |  );
```

Explanation. For total participation with minimum cardinality 1, the relationship cannot be optional. Hence, the employee entity and the relationship are merged into a single table.

Each employee record now necessarily includes the associated company. As a result, every employee is guaranteed to participate in the relationship, enforcing the $(1, 1)$ constraint.

Exception 3: Partial Key

Suppose an employee is uniquely identified only within a company. In this case, `emp_number` is a **partial key**: it distinguishes employees within the same company but not across different companies. Therefore, the employee is a weak entity that depends on the dominant entity `companies`.



Incorrect design: The schema incorrectly treats `emp_number` as a full key, instead of partial key.

```
CREATE TABLE employee_work_for (
  start_date  DATE NOT NULL,
  emp_name    VARCHAR(64) NOT NULL,
  emp_number  CHAR(8),
  email       VARCHAR(64) NOT NULL,
  name        VARCHAR(32) NOT NULL,
  PRIMARY KEY (emp_number),
  UNIQUE (email),  -- candidate key
  FOREIGN KEY (name) REFERENCES companies(name)
);
```

Correct design: Add the primary key of the dominant entity set.

```
CREATE TABLE employee_work_for (
  start_date  DATE NOT NULL,
  emp_name    VARCHAR(64) NOT NULL,
  emp_number  CHAR(8),
  email       VARCHAR(64) NOT NULL,
  name        VARCHAR(32) NOT NULL,
  PRIMARY KEY (emp_number, name),
  UNIQUE (email),  -- candidate key
  FOREIGN KEY (name) REFERENCES companies(name)
);
```

Explanation. To form a full key for a weak entity, the partial key must be combined with the primary key of the dominant entity. Here, the pair `(emp_number, name)` uniquely identifies each employee within a specific company.

5 Nested Queries

5.1 Splitting query

A **view** is an unevaluated query whose rows are not explicitly stored in the database, but is computed only when queried from a **view definition**.

```
01 | CREATE VIEW singapore_customer AS
02 |   SELECT *
03 |   FROM customers c
04 |   WHERE c.country = 'Singapore';
```

A **common table expression** (CTE) is a copy of a subquery in a temporary table that only exists for the query. Outside the query, the table does not exist.

The `WITH` clause defines a CTE; it consists of the CTE name followed by a query that produces the temporary result set.

The main query that follows can reference the CTE name as if it were a table.

```
01 | WITH singapore_customer AS
02 |   ( SELECT *
03 |     FROM customers c
04 |     WHERE c.country = 'Singapore' )
05 | SELECT cs.last_name, d.name
06 | FROM singapore_customer cs, downloads d
07 | WHERE cs.customerid = d.customerid;
```

Since the result of a subquery is a table, we can use a subquery in the `FROM` clause.

```
01 | SELECT cs.last_name, d.name
02 | FROM ( SELECT *
03 |        FROM customers c
04 |        WHERE c.country = 'Singapore' ) AS cs, downloads d
05 | WHERE cs.customerid = d.customerid;
```

Remark. As a good practice, rename the result using `AS` operator.

A single value can be treated as a table containing exactly one row and one column. Accordingly, a subquery may appear in the `SELECT` clause provided it returns exactly one row and one column; such a subquery is called a **scalar subquery**.

```
01 | SELECT (
02 |   SELECT COUNT(*) FROM customers c
03 |   WHERE c.country = 'Singapore'
04 | );
```

5.2 Nested query

If a subquery produces a one-column table with multiple rows, we can view it as a tuple of values, and thus nest it.

```
01 | SELECT d.name
02 | FROM downloads d
03 | WHERE d.customerid IN (
04 |   SELECT c.customerid
05 |   FROM customers c
```

```
06 |     WHERE c.country = 'Singapore'
07 |   );
```

Logically, it is equivalent to the following steps:

(1) Compute the `c.customerid` in the inner query.

(2) For each `d.customerid`, check that it is equal to one of the precomputed `c.customerid` values.

`IN` is logically equivalent to the comparison `= ANY` applied to the same subquery. Both expressions evaluate to true if the given value matches **at least one element** in the subquery.

```
01 |  SELECT d.name
02 |  FROM downloads d
03 |  WHERE d.customerid = ANY (
04 |    SELECT c.customerid
05 |    FROM customers c
06 |    WHERE c.country = 'Singapore'
07 |  );
```

Remark. Never use comparison to a subquery without specifying the quantifier `ALL` or `ANY`.
Without `ALL` or `ANY`, the subquery must be a scalar subquery.

`ALL` requires the comparison to hold for every value produced by the subquery. Formally, an expression of the form `x <op> ALL (subquery)` evaluates to true only if the comparison is satisfied for **all elements** in the result set.

```
01 |  -- We want to find the most expensive game.
02 |  -- But we cannot use aggregate function in the WHERE clause
03 |
04 |  -- Strategy:
05 |  -- 1. Compute the maximum price in a separate query.
06 |  -- 2. Compare each game's price against that value.
07 |
08 |  SELECT g1.name, g1.version, g1.price
09 |  FROM games g1
10 |  WHERE g1.price = ALL (
11 |    SELECT MAX(g2.price)
12 |    FROM games g2
13 |  );
```

Remark. The subquery returns exactly one value, so it is a scalar subquery. Therefore, the keyword `ALL` is not strictly necessary.

`EXISTS` evaluates to true if the subquery has some result (non-empty), false if the subquery has no result (empty).

```
01 |  SELECT d.name
02 |  FROM downloads d
03 |  WHERE EXISTS (
04 |    SELECT c.customerid
05 |    FROM customers c
06 |    WHERE d.customerid = c.customerid
07 |      AND c.country = 'Singapore'
08 |  );
```

Conceptually, evaluation can be understood using a **nested-loop model**: for each row considered by the outer query, the subquery is evaluated to check whether a matching row exists. If a match is found, the predicate is satisfied, so the outer row is retained.

Definition 5.1. A subquery is **correlated** when it refers to columns from a table in the outer query.

In this case, the column `d.customerid` from the outer query's `downloads` table appears in the `WHERE` clause of the inner query.

In general, any subquery may be written in correlated form if it depends on values from the outer query.

This follows the principle of **nested scoping**: an inner query may reference table aliases defined in an outer query, but not the other way around – the outer query cannot reference aliases defined within the inner query.

```
01 | -- find the most expensive versions of each game
02 | SELECT g1.name, g1.version, g1.price
03 | FROM games g1
04 | WHERE g1.price >= ALL (
05 |     SELECT g2.price
06 |     FROM games g2
07 |     WHERE g1.name = g2.name -- used table alias from outer query
08 | );
```

In this query, the outer and inner queries are correlated.

- The value of `g1.name` is obtained at Line 7 is obtained from the outer `FROM` clause.
- The value of `g2.name` is obtained at Line 7 is obtained from the inner `FROM` clause.

We can use subquery in `SELECT` clause, it still needs to be a scalar subquery, but it can be correlated.

```
01 | SELECT (
02 |     SELECT c.last_name
03 |     FROM customers c
04 |     WHERE c.country = 'Singapore'
05 |       AND d.customerid = c.customerid
06 |   ), d.name
07 | FROM downloads d;
```

For each row in `downloads` table, the inner query searches `customers` table for a row where `country = 'Singapore'` and `c.customerid` matches the current outer row's `customerid`.

Since `d.customerid` is unique in `downloads`, the inner query will return at most one value, making it a scalar subquery.

Nested queries are incredibly powerful when combined with **negation**.

```
01 | -- Find the customers who have not downloaded anything
02 | SELECT c.customerid FROM customers c
03 | WHERE c.customerid NOT IN (
04 |   SELECT d.customerid FROM downloads d
05 | );
```

When we want to perform **aggregation** over **two different groupings**, a single `GROUP BY` is insufficient, because each aggregation requires its own grouping context. In such cases, a nested query is needed:

- Inner query: Computes the first aggregation based on the first grouping.
- Outer query: Performs the second aggregation, grouping the results of the inner query according to a different criterion.

```
01 | -- Find the country with most customers
02 | -- 1. For each country, count the number of customers
```

```sql
-- 2. Compare each country's count against the counts of all countries to
    find the maximum
SELECT c1.country
FROM customers c1
GROUP BY c1.country
HAVING COUNT(*) >= ALL (
  SELECT COUNT(*)
  FROM customers c2
  GROUP BY c2.country
);
```

This is the typical query for **universal quantification**:

$$\forall x f(x) \equiv \neg \exists x \neg f(x)$$

```sql
-- Students who borrowed all the books authored by Adam Smith
SELECT s.email, s.name
FROM student s
WHERE NOT EXISTS (
  SELECT *
  FROM book b
  WHERE authors = 'Adam Smith'
    AND NOT EXISTS (
      SELECT *
      FROM loan l
      WHERE l.book = b.ISBN13
      AND l.borrower = s.email));
```

Define the predicates

$$\begin{aligned} \text{Book}(x) &\equiv (x \text{ is a book authored by Adam Smith}), \\ \text{Loan}(s, x) &\equiv (s \text{ borrowed } x). \end{aligned}$$

We are asked to find students s that satisfy

$$\forall x[\text{Book}(x) \rightarrow \exists l \ \text{Loan}(s, x)].$$

By universal quantification, this is equivalent to

$$\neg \exists x[\text{Book}(x) \wedge \neg \exists l \ \text{Loan}(s, x)].$$

Equivalently, the student satisfies the condition: “There does not exist a book authored by Adam Smith that the student did not borrow”.

6 Relational Algebra

To come up with a relational algebra expression to solve a given query:

(1) Draw out all tables and their columns

(2) Identify the tables relevant to the problem

(3) Come up with a solution

(4) Translate into relational algebra

Remark. Nested relational algebras are not allowed.

6.1 Unary operators

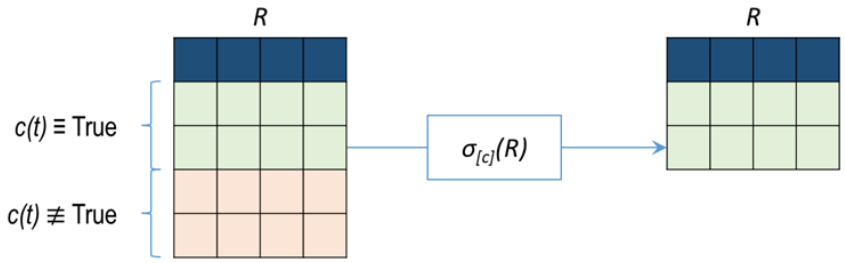
Selection

Definition 6.1. The **selection operator** $\sigma_c(R)$ selects all tuples/rows t from the relation R that satisfies the selection condition c.

The subscript c specifies the selection criterion to be applied while retrieving tuples, i.e., tuples t such that $c(t)$ is true.
This is similar to a `WHERE` clause in SQL.

Example. Let R denote an instance of the Sailors relation. We can retrieve rows corresponding to expert sailors using the expression

$$\sigma_{\text{rating} > 8}(R).$$

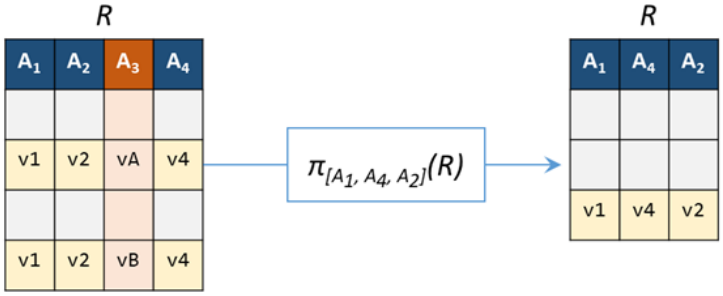


Projection

Definition 6.2. The **projection operator** $\pi_l(R)$ keeps only the columns specified in the ordered list l, and in the same order.

This is similar to `SELECT` clause in SQL, but cannot add more columns. Duplicate rows are automatically deduplicated.

Example. We can find out all sailor names and ratings using the expression $\pi_{[\text{sname, rating}]}(R)$, where the subscript specifies the attributes to be retained.



Since the result of a relational algebra expression is always a relation, we can substitute an expression wherever a relation is expected.

Example. We can compute the names and ratings of highly rated sailors by combining two of the preceding queries:

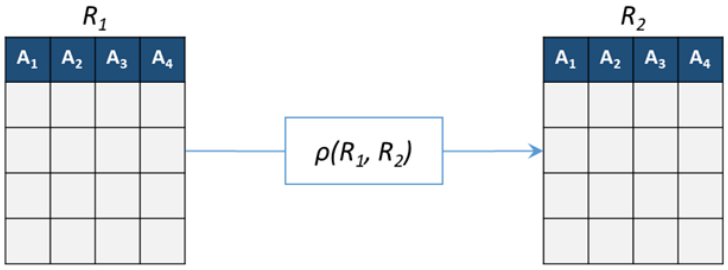
$$\pi_{[\text{sname, rating}]}(\sigma_{\text{rating} > 8}(R)).$$

We first apply the selection to R, and then apply the projection to the resulting relation.

Renaming

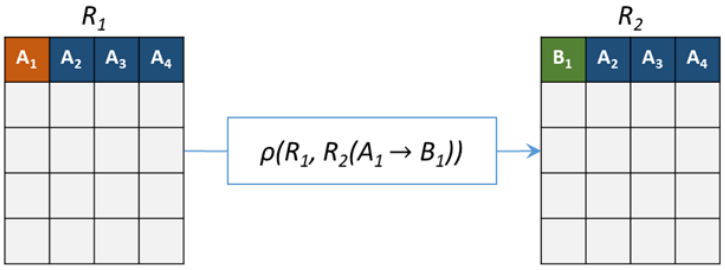
Definition 6.3. The **renaming operator** $\rho(R_1, R_2)$ is used to rename the relation R_1 (old name) as R_2 (new name). We can now refer to R_1 as R_2, but only within the query.

This is similar to table alias in `FROM` clause in SQL.



Not only can we rename the relation, we can also rename the attributes:

Definition 6.4. $\rho(R_1, R_2(A_1 \rightarrow A_2))$ can be used to rename the relation and some of its attributes. The attribute $R_1.A_1$ can now be referred as $R_2.A_2$ but only within the query.



6.2 Binary operators

Set

Definition 6.5. Two relation instances are said to be **union-compatible** if

(i) they have the same number of the fields, and

(ii) corresponding fields, taken in order from left to right, have the same domains.

Definition 6.6. Let R and S be two relations that are union-compatible.

- The **union** $R \cup S$ returns a relation containing all tuples that occur in either R or S (or both).
- The **intersection** $R \cap S$ returns a relation containing all tuples that occur in both R and S.
- The **set difference** $R - S$ returns a relation containing all tuples that occur in R but not in S.

In SQL, the commands are `UNION`, `INTERSECT`, and `EXCEPT`.

Product

Definition 6.7. The **cross product** (or **Cartesian product**) $R \times S$ returns a relation containing one tuple (r, s) (the concatenation of tuples r and s) for each pair of tuples $r \in R$, $s \in S$.

Properties:

- The number of columns is $n + m$.
- The number of rows is $n \times m$.

Join

The join operation is used to combine information from two or more relations.

Definition 6.8. The **join operator** is defined to be a cross-product followed by a selection:

$$R \bowtie_c S \coloneqq \sigma_c(R \times S).$$

In other words, we include only tuples that satisfy the condition c after the cross product.

Definition 6.9. The **equijoin operator** $R \bowtie S$ is when the join condition solely consists of equalities (connected by $\wedge$) of the form

$$R.name1 = S.name2$$

i.e., equalities between two attributes in R and S.

Definition 6.10. The **natural join operator** $R \bowtie S$ is an equijoin in which equalities are specified on all fields having the same name in R and S.

In this case, we simply omit the join condition.
The natural join has the nice property that the result is guaranteed not to have two fields with the same name.

Remark. If two relations have no attributes in common, then $R \bowtie S$ is simply the cross product.

Whereas the result of a join (or inner join) consists of tuples formed by combining matching tuples in the two operands, an **outer join** contains those tuples and additionally some tuples formed by extending an unmatched tuple in one of the operands by null padding the attributes of the other operand.

Definition 6.11.

- The **left outer join** $R \mathbin{⟕} S$ contains all combinations of tuples in R and S that are equal on their common attribute names, in addition to tuples in R that have no matching tuples in S.
- The **right outer join** $R \mathbin{⟖} S$ contains all combinations of tuples in R and S that are equal on their common attribute names, in addition to tuples in S that have no matching tuples in R.
- The **full outer join** $R \mathbin{⟗} S$ contains all combinations of tuples in R and S that are equal on their common attribute names, in addition to tuples in S that have no matching tuples in R, and tuples in R that have no matching tuples in S in their common attribute names.

$$R \mathbin{⟗} S = (R \mathbin{⟕} S) \cup (R \mathbin{⟖} S).$$

7 Stored Procedures and Functions

The SQL:1999 standard introduced the concept of **stored procedures and functions**. This enables creation and execution of code directly within the database. **PL/pgSQL** is the language used to write stored procedures/functions.

The difference is stored procedure has no return, stored function has return.

7.1 Stored functions

```
01 | CREATE OR REPLACE FUNCTION <fname>
02 |   ( <param> <type>,  <param> <type>, ... )
03 | RETURNS <type> AS $$
04 | DECLARE
05 |   <var> <type>;
06 |     :
07 | BEGIN
08 |   <code>
09 | END;
10 | $$ LANGUAGE <language>;
```

- Function definition is enclosed in double dollars `$$`
- Blocks open with keyword and closed with `END` .
- Function signature contains function name and parameters with types.
- Return type is declared with keyword `RETURNS` (with `S`). The actual return is given by the keyword `RETURN` (without `S`).
- Declare local variables using `DECLARE` .
- Assignment is done with `:=` , because `=` is already used for equality check.
- Selection is done with `IF .. THEN .. END IF;` . The condition is between `IF .. THEN` , and the body is between `THEN .. END IF;` . Optionally, we may have `ELSE` .
- Declare the language `plpgsql` at the end.

Invocation

We can invoke the function in a `SELECT` clause using the standard function call technique.

```
01 | SELECT calculate_age('2001-12-13');
```

Since it can be used in a `SELECT` clause, it can be used as part of an SQL query.

```
01 | SELECT c.customerid, calculate_age(c.dob) AS age
02 | FROM customers c
03 | ORDER BY age;
```

Loop

While loop uses `WHILE <condition> LOOP .. END LOOP` . Note that we need to manually increment the counter for the loop.

```
01 | CREATE OR REPLACE FUNCTION calculate_day(dob DATE)
02 | RETURNS INTEGER AS $$
03 | DECLARE
04 |   days INTEGER;
05 | BEGIN
06 |   days := 0;
07 |   WHILE dob < CURRENT_DATE LOOP
08 |     days := days + 1; -- increment integer
09 |     dob := dob + 1; -- increment date
10 |   END LOOP;
11 |   RETURN days;
12 | END;
13 | $$ LANGUAGE plpgsql;
```

Parameters

There are three types of parameters:

- `IN` : input parameter (passed into to a procedure)
- `OUT` : output parameter (returned from a procedure)
- `IN OUT` : can act as both input and output

Tuple

We can return a single tuple from existing table by specifying the table name as the return type.

```
01 | CREATE OR REPLACE FUNCTION most_expensive()
02 | RETURNS games AS $$
03 |   SELECT *
04 |   FROM games g
05 |   ORDER BY g.price DESC
06 |   LIMIT 1; -- limit to 1 row
07 | $$ LANGUAGE sql;
```

```
01 | SELECT *
02 | FROM most_expensive();
```

We can return an arbitrary single tuple by specifying `OUT` parameter and returning a `RECORD` .

```
01 | CREATE OR REPLACE FUNCTION most_download
02 |   ( OUT name VARCHAR(32), OUT version CHAR(3) )
03 | RETURNS RECORD AS $$
04 |   SELECT g.name, g.version
05 |   FROM games g, downloads d
06 |   WHERE g.name = d.name AND g.version = d.version
07 |   GROUP BY g.name, g.version
08 |   ORDER BY COUNT(*) DESC
09 |   LIMIT 1;
10 | $$ LANGUAGE sql;
```

Table

We can return a table from an existing table by specifying the table name as the return type prefixed with `SETOF`.

```
CREATE OR REPLACE FUNCTION all_most_expensive()
RETURNS SETOF games AS $$
  SELECT *
  FROM games g
  GROUP BY g.name, g.version
  HAVING g.price >= ALL(
    SELECT g1.price FROM games g1
  );
$$ LANGUAGE sql;
```

Remark. `SETOF` means multiple tuples.

We can return an arbitrary table by specifying `OUT` parameter and returning a `SETOF RECORD`.

```
CREATE OR REPLACE FUNCTION all_most_download
  ( OUT name VARCHAR(32), OUT version CHAR(3) )
RETURNS SETOF RECORD AS $$
  SELECT g.name, g.version
  FROM games g, downloads d
  WHERE g.name = d.name AND g.version = d.version
  GROUP BY g.name, g.version
  HAVING COUNT(*) >= ALL ( /* ... */ );
$$ LANGUAGE sql;
```

We can also return an arbitrary table by returning `TABLE(..)` with the attributes specified for the given table.

```
CREATE OR REPLACE FUNCTION all_most_download()
RETURNS TABLE(
  name VARCHAR(32), version
  CHAR(3))
AS $$
  ...
$$ LANGUAGE sql
```

Raise

We can create a **log message**. PL/pgSQL provide 6 different message levels. Each level has a different priority, with the highest priority allows us to abort the transaction.

- `RAISE DEBUG`
- `RAISE LOG`
- `RAISE INFO`
- `RAISE NOTICE`
- `RAISE WARNING`
- `RAISE EXCEPTION`

7.2 Stored procedures

We can think of a procedure as a function that does not return any value.

We typically use procedures to enforce certain constraint when performing insertion/deletion/update, e.g., some check is done before inserting a tuple into a table.

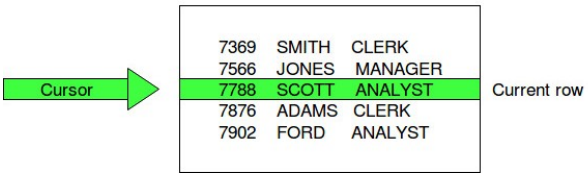
```
CREATE OR REPLACE PROCEDURE <pname>
  ( <param> <type>, <param> <type>, ... )
AS $$
DECLARE
  <var> <type>;
    :
BEGIN
  <code>
END;
$$ LANGUAGE <language>;
```

Example.

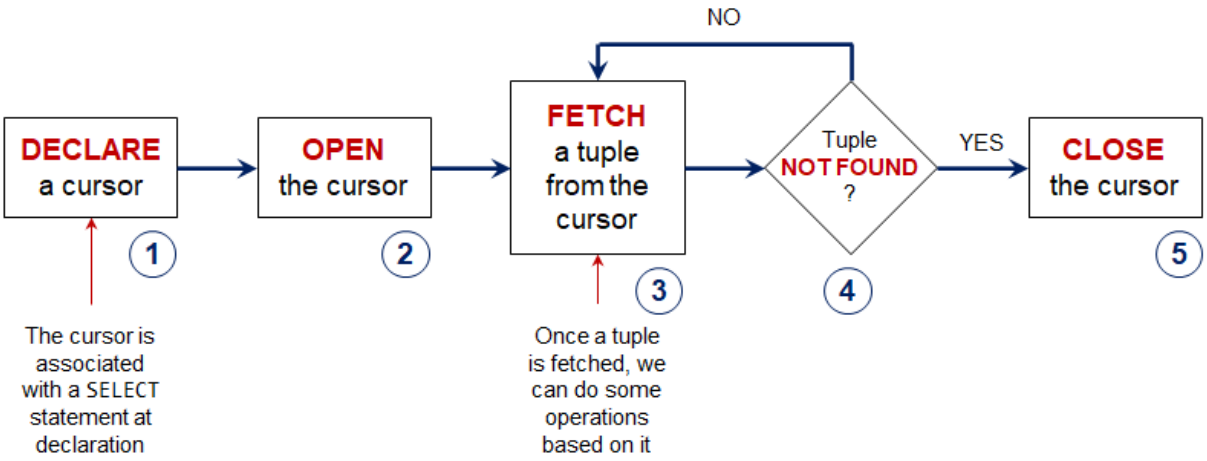
```
CREATE OR REPLACE PROCEDURE download_game
  ( cid VARCHAR(16), gname VARCHAR(32), gver CHAR(3) )
AS $$
BEGIN
  IF NOT EXISTS ( -- check for violation
    SELECT c.customerid FROM customers c
    WHERE gname = 'Domainer' AND calculate_age(c.dob) < 21
      AND c.customerid = cid ) THEN
    INSERT INTO downloads VALUES (cid, gname, gver);
  END IF;
END; $$ LANGUAGE plpgsql;
```

7.3 Cursor

PL/pgSQL offers a **cursor** mechanism, which allow us to process query results one row at a time, i.e., iterate over the rows.



The following is a workflow of steps to use a cursor.



(1) Declare a cursor with type `CURSOR`.

The cursor is always associated with a query that determines the result set, using the `FOR` keyword. This query may optionally take in parameters that will be supplied later during execution.

(2) Open the cursor using the `OPEN` keyword.

If the cursor was defined with parameters, then the corresponding arguments are passed in at this stage.

(3) Once opened, the program repeatedly retrieves rows from the cursor and processes them. This is typically done inside a loop.

Each iteration uses the `FETCH` keyword to obtain the next row, and assign it to a local variable `curr`. By default, `FETCH` advances to the next row in the result set.

(4) After each fetch, a condition is checked to determine whether a row was successfully retrieved. If no row is found, it indicates that the cursor has reached the end of the result set. This is commonly handled using `EXIT WHEN NOT FOUND` to terminate the loop.

(5) Finally, close the cursor using the `CLOSE` keyword.

```
01 | -- Find the largest increase in price between versions
02 | CREATE OR REPLACE FUNCTION max_increase(gname VARCHAR(32))
03 | RETURNS NUMERIC AS $$
04 | DECLARE
05 |   cur CURSOR (vname VARCHAR(32)) FOR
06 |     SELECT g.price
07 |     FROM games g
08 |     WHERE g.name = vname
09 |     ORDER BY g.version ASC;
10 |   res NUMERIC;
11 |   prev NUMERIC;
12 |   curr NUMERIC;
13 | BEGIN
14 |   OPEN cur(gname);
15 |   res := 0; prev := 0;
16 |   LOOP
17 |     FETCH cur INTO curr;
18 |     EXIT WHEN NOT FOUND;
19 |     IF (curr - prev) >= res
20 |     THEN res := (curr - prev);
21 |     END IF;
22 |     prev := curr;
```

```
23 |     END LOOP;
24 |     CLOSE cur;
25 |     RETURN res;
26 |   END; $$ LANGUAGE plpgsql;
```

Direction

A helpful way to understand the behavior of `FETCH` is to view it as performing two actions simultaneously:

(1) **Move** the cursor to a new location.

(2) **Retrieve** the row at the new location.

When a cursor is first opened, it is positioned just before the first row of the result set. Any subsequent `FETCH` operation will therefore determine both where the cursor moves and which row is returned.
Different directional options specify how the cursor should move.

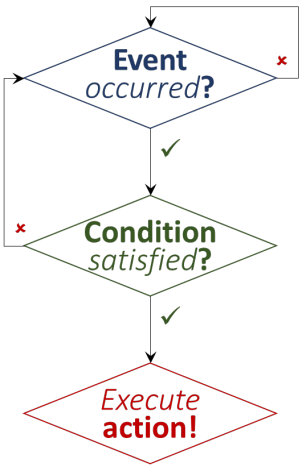
- `NEXT` / `FORWARD` : To the next row (default)
- `PRIOR` / `BACKWARD` : To the previous row
- `FIRST` : To the first row
- `LAST` : To the last row
- `ABSOLUTE n` : To the index n (starting from 0, can be negative)
- `RELATIVE n` : Forward by n rows (or backward if negative)

8 Triggers

8.1 Definition

Definition 8.1. A **trigger** is a procedure or function executed when a database event (insert/delete/update) occurs on a table.

A trigger is usually used to enforce some complicated constraint that cannot be enforced using a simple `CHECK`.



A trigger consists of two components.

(1) The **trigger function** specifies the action that is executed when the event occurs (what to do).

The function returns a trigger, i.e., a value of type `TRIGGER`.

It typically checks for violations using an SQL query. If a violation is detected, it raises an exception to roll back the operation. Otherwise, it returns `NEW` to allow the operation to proceed.

(2) The **trigger** determines when it should fire by binding a trigger function to operations on a table.

We need to specify

(a) event (insertion / update / delete),

(b) timing (before / after),

(c) granularity (row / statement).

```
01 | CREATE TRIGGER <tname>
02 | <timing> <event> ON <table>
03 | FOR EACH <granularity>
04 |   [WHEN <condition>]
05 |   EXECUTE FUNCTION <fname>();
```

Example. Consider

```
01 | -- Trigger function
02 | CREATE OR REPLACE FUNCTION fr21()
03 | RETURNS TRIGGER AS $$
04 | BEGIN
05 |   IF EXISTS (
06 |     SELECT c.customerid
07 |     FROM customers c NATURAL JOIN downloads d
08 |     WHERE d.name = 'Domainer'
```

```
09 |       AND EXTRACT(year FROM AGE(c.dob)) < 21
10 |     ) THEN
11 |       RAISE EXCEPTION 'Underaged!'; -- STOP!
12 |     END IF;
13 |     RETURN NEW;
14 |   END;
15 |   $$ LANGUAGE plpgsql;
```

```
01 |   -- Trigger
02 |   CREATE CONSTRAINT TRIGGER tr21
03 |   AFTER INSERT OR UPDATE ON downloads
04 |   DEFERRABLE INITIALLY DEFERRED -- deferrable: checked only at the end of the
       transaction
05 |   FOR EACH ROW
06 |   EXECUTE PROCEDURE fr21();
```

8.2 Syntax

Transition variables

The keywords `NEW` and `OLD` are called **transition variables**.

- `NEW` refers to the modified row after the triggering event.
- `OLD` refers to the modified row before the triggering event.

The availability of transition variables depends on the type of operation:

	NEW	OLD
INSERT	yes	no
UPDATE	yes	yes
DELETE	no	yes

Timing

Triggers can be defined relative to the timing of the event.

- A `BEFORE` trigger allows us to validate or modify the tuple **before** the operation takes place.

  It corresponds to a **local check**, where we check if each operation will violate constraints before execution.

  Note: For row-Level `BEFORE` triggers, returning `NULL` signals the database to skip the operation for that specific row.

- An `AFTER` trigger checks the state of the database **after** the operation has been executed.

  It corresponds to a **global check**, where we perform operation and then check if there are any inconsistent rows at the end.

Return

The **return value** of a trigger function determines how the operation proceeds.

- For `BEFORE` triggers, returning `NEW` allows the operation to proceed (possibly with modifications to the tuple being inserted/updated); returning `NULL` cancels the operation for that tuple.

  Returning `OLD` is generally not meaningful for `INSERT` .

- For `AFTER` triggers, the return value has no effect.

	`NULL` returned	Non-`NULL` returned
`BEFORE INSERT`	No tuple inserted	Tuple inserted
`BEFORE UPDATE`	No update	Tuple updated
`BEFORE DELETE`	No deletion	Deletion proceeds
`AFTER` trigger	No effect	No effect

Granularity

Triggers can be defined at different granularities.

- A **row-level trigger** executes once for each affected tuple.

  The keyword is `FOR EACH ROW`.

- A **statement-level trigger** executes once per SQL statement, regardless of how many rows are affected.

  The keyword is `FOR EACH STATEMENT`.

e.g. INSERT can have multiple rows (multiple tuples inserted in a single insert statement)
Row-level triggers have access to `NEW` and `OLD`, whereas statement-level triggers do not (since there are multiple rows, dont know NEW refers to what). Furthermore, statement-level triggers are executed even if the statement affects zero rows.

```
01 | CREATE OR REPLACE FUNCTION no_delete()
02 | RETURNS TRIGGER AS $$
03 | BEGIN
04 |   RAISE EXCEPTION 'No delete from log...';
05 | END; $$ LANGUAGE plpgsql;
06 |
07 | CREATE TRIGGER warn_delete
08 | BEFORE DELETE ON downloads_log
09 | FOR EACH STATEMENT
10 | EXECUTE FUNCTION no_delete();
```

Order

There can be multiple triggers defined for the same event on the same table. The order of activation is:

(1) `BEFORE` statement-level trigger

(2) `BEFORE` row-level trigger

(3) `AFTER` row-level trigger

(4) `AFTER` statement-level trigger

Within each category, triggers are executed in **alphabetical order** of their names.

Logging operations

We may want to record all operations performed on a table. This can be done by defining a trigger on `INSERT`, `UPDATE`, and `DELETE`, and inserting a corresponding record into a log table.

```
01 | CREATE OR REPLACE FUNCTION log_ops()
02 | RETURNS TRIGGER AS $$
03 | BEGIN
```

```
04 |    INSERT INTO downloads_log VALUES
05 |      (NEW.customerid, NEW.name, TG_OP, CURRENT_DATE);
06 |    RETURN NEW;
07 |  END;
08 |  $$ LANGUAGE plpgsql;
09 |
10 |  CREATE TRIGGER op_log
11 |  AFTER INSERT OR DELETE OR UPDATE
12 |  ON downloads
13 |  FOR EACH ROW
14 |  EXECUTE FUNCTION log_ops();
```

`TG_OP` is a text that stores the type of operation for which the trigger was fired.

9 Functional Dependencies

9.1 Definition

Definition 9.1. Let R be a relation schema, and let X and Y be non-empty sets of attributes in R. We say that an instance r of R satisfies the **functional dependency** $X \to Y$ if the following holds for every pair of tuples $t_1, t_2 \in r$:

$$\text{If } t_1.X = t_2.X, \quad \text{then } t_1.Y = t_2.Y.$$

We say that X **(functionally) determines** Y.

That is, if two tuples agree on the values in attributes X, they must also agree on the values in attributes Y.

9.2 Reasoning

Definition 9.2. We say that an FD f is **implied by** a given set F of FDs if f holds on every relation instance that satisfies all dependencies in F, that is, f holds whenever all FDs in F hold.

Definition 9.3. The set of all FDs implied by a given set F of FDs is called the **closure** of F, denoted as F^+.

An important question is how we can infer, or compute, the closure of a given set F of FDs. The following three rules, called **Armstrong's Axioms**, can be applied repeatedly to infer all FDs implied by a set F of FDs.
We use X, Y, and Z to denote sets of attributes over a relation schema R:

Axiom 9.4 (Armstrong's axioms)**.**

(1) Axiom of reflexivity: If $X \supseteq Y$, then $X \to Y$.

(2) Axiom of augmentation: If $X \to Y$, then $XZ \to YZ$ for any Z.

(3) Axiom of transitivity: If $X \to Y$ and $Y \to Z$, then $X \to Z$.

It is convenient to use some additional rules while reasoning about F^+:

Lemma 9.5. *Rule of decomposition: If $X \to YZ$, then $X \to Y$ and $X \to Z$.*

Proof. By reflexivity, we have $YZ \to Y$ and $YZ \to Z$.
By transitivity, we have $X \to YZ$ and $YZ \to Z$ imply $X \to Y$. Similarly, $X \to YZ$ and $YZ \to Z$ imply $X \to Z$. □

Lemma 9.6. *Rule of union: If $X \to Y$ and $X \to Z$, then $X \to YZ$.*

Proof. By augmentation, $X \to Y$ implies $X \to XY$.
By augmentation, $X \to Z$ implies $XY \to YZ$.
By transitivity, $X \to XY$ and $XY \to YZ$ imply $X \to YZ$. □

By definition, to prove that $X \to Y$ holds, we need to show that $Y \in X^+$.

Algorithm 9.7 (Compute closure)**.** *Given $A_1, A_2, \ldots, A_n$, the closure $\{A_1, A_2, \ldots, A_n\}^+$ can be computed as follows:*

(1) Initialise the closure to $\{A_1, A_2, \ldots, A_n\}$.

(2) If there is a FD $A_i, A_j, \ldots, A_m \to B$ such that $A_i, A_j, \ldots, A_m$ are all in the closure, then put B into the closure.

(3) Repeat Step 2, until we cannot find any new attribute to put into the closure.

9.3 Keys, superkeys, prime attributes

Definition 9.8. If $X \to Y$ holds, where Y is the set of all attributes, then X is a **superkey**.

Definition 9.9. A **key** is a minimal superkey.

Remark. Minimality means that if we remove any attribute from the key, then it will not be a key anymore.

Remark. A superkey is a superset of a key.

By definition, the following is a trivial algorithm to find keys of a table.

Algorithm 9.10 (Finding keys)**.** *Given a table R,*

(1) Consider every subset of attributes in R.

(2) Derive the closure of each subset.

(3) Identify superkeys based on the closures.

(4) Identify keys from the superkeys.

Tip 1: If an attribute does not appear on the right-hand side, then it must be in the key (since no other attribute can decide it).

Tip 2: Check small attribute sets first, i.e., start from sizes 1, 2, and so on.

Remark. Before step (1), may need to simplify FDs to remove trivial attributes from FDs.

Example. Consider $R(A, B, C, D, E)$, with FDs $AB \to C$, $C \to B$, $BC \to D$, $CD \to E$. Note that A must be in every key. We compute the closures:

$$A^+ = A, \quad AB^+ = ABCDE, \quad AC^+ = ACBDE, \quad AD^+ = AD, \quad AE^+ = AE, \quad ADE^+ = ADE.$$

At this point, we can stop. Hence, the possible keys are AB and AC.

Definition 9.11. If an attribute appears in a key, then it is a **prime attribute**.

10 Boyce-Codd Normal Form

10.1 Definitions

Let R be a relation schema, and let X be a subset of attributes of R.

Definition 10.1. An FD $X \to A$ is called **decomposed** if A consists of a single attribute.

Remark. A non-decomposed FD can always be transformed into an equivalent set of decomposed FDs, using the rule of decomposition. For instance, $BC \to DE$ is equivalent to $BC \to D$ and $BC \to E$.

Definition 10.2. An FD $X \to A$ is called **trivial** if $A \in X$. Otherwise, the FD is called **non-trivial**.

Non-trivial means not implied by axiom of reflexivity.
Non-trivial and decomposed FDs can be found using closures, in a way similar to finding keys.

(1) Consider all attribute subsets in R.

(2) Compute the closure of each subset.

(3) From each closure, remove the "trivial" attributes.

(4) Derive non-trivial and decomposed FDs from each closure.

Example. Given $R(A, B, C)$ and FDs $A \to B$, $B \to A$, $B \to C$, first compute the closure of each attribute subset:

$$A^+ = ABC, \quad B^+ = ABC, \quad C^+ = C, \quad AB^+ = ABC, \quad AC^+ = ABC, \quad BC^+ = ABC.$$

From each closure, remove the "trivial" attributes:

$$A^+ = BC, \quad B^+ = AC, \quad AB^+ = C, \quad AC^+ = B, \quad BC^+ = A.$$

Finally, derive non-trivial and decomposed FDs from each closure:

$$A \to B, \quad A \to C, \quad B \to A, \quad B \to C, \quad AB \to C, \quad AC \to B, \quad BC \to A.$$

Definition 10.3. We say that R is in **Boyce-Codd normal form** (BCNF) if for every non-trivial and decomposed FD $X \to A$, X is a superkey.

In practical terms, BCNF requires that if there exists any non-trivial and decomposed FD of the form $A_1A_2 \cdots A_n \to B$, then the set of attributes $\{A_1, A_2, \ldots, A_n\}$ must be a superkey. Put simply, all non-trivial dependencies must originate from a superkey; attributes should depend on "the key, the whole key, and nothing but the key."
Why is this constraint necessary? To understand the logic, consider a scenario where an attribute B depends on a set of attributes $C_1C_2 \cdots C_n$ that is *not* a superkey:

$$C_1C_2 \cdots C_n \to B$$

Since $\{C_1, C_2, \ldots, C_n\}$ is not a superkey, the same combination of values for these attributes can appear in multiple distinct rows within the table. Due to the functional dependency, every time this combination $C_1C_2 \cdots C_n$ repeats, the corresponding value of B must also repeat. This creates unnecessary data redundancy.
By enforcing the BCNF requirement, we ensure that such dependencies only exist on superkeys (which are unique for every row), thereby eliminating redundancy.

Example. In the above example where $R(A, B, C)$ and FDs $A \to B$, $B \to A$, $B \to C$, note that the keys are A and B. Thus, for each of the non-trivial and decomposed FDs, the left-hand side is a superkey. Hence, R satisfies BCNF.

10.2 BCNF check

How do we check whether a table is in BCNF? Based on definition alone, the following is a brute force algorithm for checking BCNF:

(1) Compute the closure of each attribute subset

(2) Derive the keys of R (using closures)

(3) Derive all non-trivial and decomposed FDs on R (again, using closures)

(4) Check the non-trivial and decomposed FDs to see if they satisfy the BCNF requirement

To simplify the check, we focus on identifying a specific failure condition: a non-trivial and decomposed FD $A_1A_2 \dots A_k \rightarrow B$ where $\{A_1, A_2, \dots, A_k\}$ is *not* a superkey.
If a subset of attributes $X = \{A_1, A_2, \dots, A_k\}$ is not a superkey, its closure X^+ will exhibit two specific characteristics:

- **More:** X^+ contains at least one attribute B that is not in X, so X^+ contains more attributes than X.
- **Not All:** The closure X^+ does not contain every attribute in the relation R (since X is not a superkey).

A relation violates BCNF if and only if there exists a subset of attributes whose closure satisfies the **more but not all** condition.
Conversely, if every subset's closure results in either exactly itself (trivial) or the set of all attributes (superkey), the table is in BCNF.

Algorithm 10.4 (BCNF check)**.**

(1) Compute the closure of each attribute subset.

(2) Check if there is a "more but not all" closure.

(3) If such a closure exists, then R is not in BCNF.

Example. Consider $R(A, B, C, D)$ with FDs $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$.

(1) Compute the closure of each attribute subset:

$$A^+ = A, \quad B^+ = B, \quad C^+ = ACD, \quad D^+ = AD.$$

(2) Notice C^+ contains more attributes than C, but not all attributes in R.

(3) This is a violation of BCNF. Hence, R is not in BCNF.

10.3 Decomposition algorithm

If a table is not in BCNF, we can decompose it into smaller tables in BCNF.

Definition 10.5. A **decomposition** of a relation schema R consists of replacing the relation schema by two (or more) relation schemas that each contain a subset of the attributes of R, and together include all attributes in R.

We now present an algorithm for decomposing a relation schema R into a collection of BCNF relation schemas:

Algorithm 10.6 (Decomposition algorithm)**.** *Suppose that R is not in BCNF.*

(1) Find a subset X of attributes of R, such that X^+ contains more attributes than X, but not all attributes in R.

(2) Decompose R into two tables R_1 and R_2, such that

- *R_1 contains all attributes in X^+,*

- *R_2 contains all attributes in X as well as attributes not in X^+.*

Note that the common link between two tables is the attributes in X.

(3) If R_1 is not in BCNF, further decompose R_1.
If R_2 is not in BCNF, further decompose R_2.

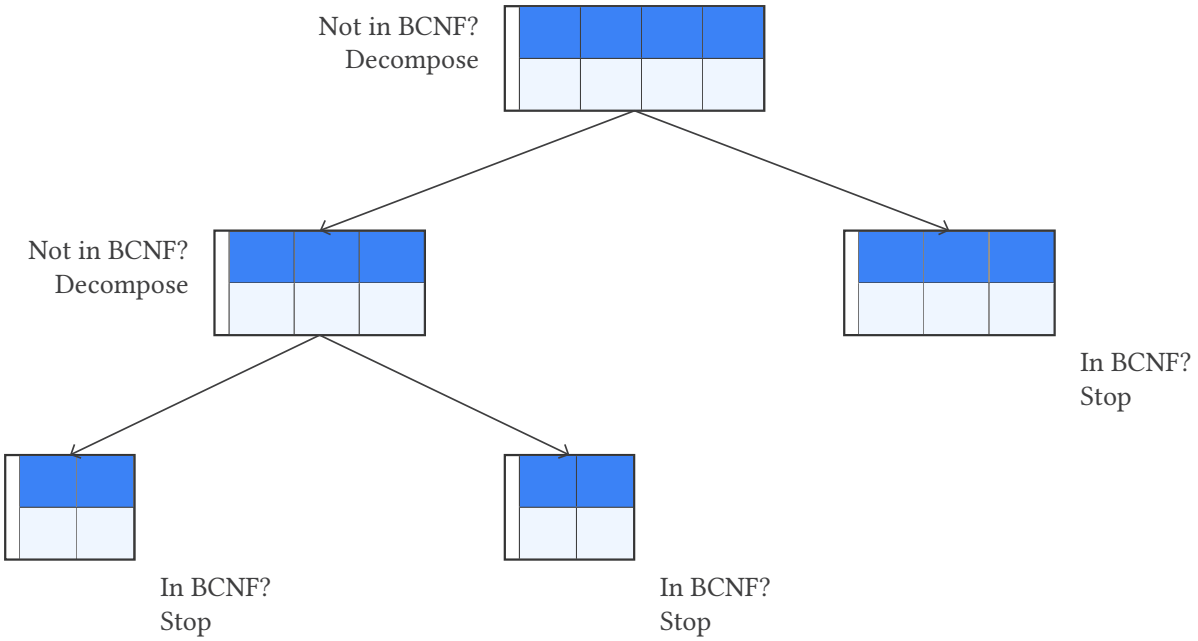


Figure 1: BCNF decomposition.

Example. Consider $R(A, B, C, D)$ with FDs $A \to B$, $B \to C$.

- $A^+ = ABC$, so decompose R into $R_1(A, B, C)$ and $R_2(A, D)$.
- $B^+ = BC$, so further decompose R_1 into $R_3(B, C)$ and $R_4(A, B)$.

Lemma 10.7. *Every relation schema with exactly two attributes is in BCNF.*

Proof. Let $R(A, B)$ be a relation schema. We examine all possible non-trivial, decomposed FDs:

- Case 1: No non-trivial FDs exist. Then BCNF is satisfied vacuously.
- Case 2: $A \to B$. Then $A^+ = AB$, so A is a key (and thus a superkey).
- Case 3: $B \to A$. Similarly, $B^+ = AB$, so B is a key.
- Case 4: $A \leftrightarrow B$ ($A \to B$ and $B \to A$). Then both A and B are keys.

Since every determinant in a non-trivial FD is a superkey, R is in BCNF. □

When a relation R is decomposed into smaller relations $R_1, R_2, \ldots, R_n$, the FDs that held on the original table must be **projected** onto the new schemas. This is crucial because an FD that wasn't explicitly listed for a sub-schema might still be implied by the original set of dependencies.
In general, to derive the closures on a table R_i decomposed from a table R,

(1) Enumerate the attribute subsets of R_i.

(2) For each subset, derive its closure on R.

(3) Project each closure onto R_i by removing those attributes that do not appear in R_i.

These projected closures can then be used to decide whether R_i is in BCNF.

A decomposition is only valid if we can get the original data back without gaining "phantom" or "spurious" tuples.

Definition 10.8. Let R be a relation schema, and let F be a set of FDs over R. A decomposition of R into two schemas with attribute sets X and Y is a **lossless-join decomposition** if for every instance r of R that satisfies the dependencies in F,

$$\pi_X(r) \bowtie \pi_Y(r) = r.$$

All decompositions used to eliminate redundancy must be lossless. The following test is very useful:

Lemma 10.9. *A decomposition of R into R_1 and R_2 is lossless if and only if F^+ contains either the FD $R_1 \cap R_2 \longrightarrow R_1$ or $R_1 \cap R_2 \longrightarrow R_2$.*

That is, the common attributes in R_1 and R_2 constitute a superkey of R_1 or R_2.

Example. Consider $R(A, B, C)$ with FD $B \longrightarrow C$.

Since B is not a superkey, decompose R into $R_1(B, C)$ and $R_2(A, B)$.

The intersection is $R_1 \cap R_2 = \{B\}$. Since $B \longrightarrow C$ holds, B is a superkey for R_1, so $B \longrightarrow R_1$. Hence, the decomposition is lossless.

However, a BCNF decomposition may not always preserve dependencies. Consider $R(A, B, C)$ with FDs $AB \longrightarrow C$, $C \longrightarrow B$. Since $C^+ = BC$ is a "more but not all" closure, R is not in BCNF. Thus, decompose R into $R_1(B, C)$ and $R_2(A, C)$.

The non-trivial and decomposed FD on R_1 is $C \longrightarrow B$, but no non-trivial and decomposed FDs on R_2. The other FD $AB \longrightarrow C$ cannot be derived from the FDs on R_1 and R_2, i.e., it is "lost".

Definition 10.10. Let F be the given set of FDs on the original table. Let F' be the set of FDs on the decomposed tables. We say that the decomposition **preserves** all FDs if F and F' are **equivalent**, i.e.,

- Every FD in F' can be derived from F ($F \implies F'$).
- Every FD in F can be derived from F' ($F' \implies F$).

Example. Consider $F = \{A \longrightarrow B, B \longrightarrow C, A \longrightarrow C\}$ and $F' = \{A \longrightarrow B, B \longrightarrow C\}$. Obviously F' can be derived from F. On the other hand, F can be derived from F', since $A \longrightarrow B$ and $B \longrightarrow C$ imply $A \longrightarrow C$. Hence, F and F' are equivalent.

11 Third Normal Form

11.1 Definitions

Let R be a relation schema, and let X be a subset of attributes of R.

Definition 11.1. We say that R is in **third normal form** (3NF) if for every non-trivial and decomposed FD $X \to A$,

(1) either X is a superkey, or

(2) A is a prime attribute.

Example. Consider $R(A, B, C)$ with non-trivial and decomposed FDs $C \to B$, $AC \to B$, $AB \to C$. The keys are AB, AC.

- $C \to B$ is OK, since B is a prime attribute.
- $AC \to B$ is OK, since AC is a key of R.
- $AB \to C$ is OK, since AB is a key of R.

Hence, R is in 3NF.

Example. Consider $R(A, B, C)$ with non-trivial and decomposed FDs $A \to B$, $B \to C$, $AC \to B$, $AB \to C$. The key is A.
Notice $B \to C$ is not OK, since B is not a superkey of R, *and* C is not a prime attribute. Hence, R is NOT in 3NF.

3NF is more "lenient" than BCNF. Therefore, satisfying BCNF $\implies$ satisfying 3NF, but not necessarily vice versa.
Violating 3NF $\implies$ violating BCNF, but not necessarily vice versa.

11.2 3NF check

Based on the definition of 3NF, one can deduce the following straightforward algorithm to check if a relation schema R is in 3NF.

Algorithm 11.2 (3NF check)**.**

(1) Compute the closure for each subset of the attributes in R, find the keys.

(2) For each given FD, check if left-hand side is a superkey, or each attribute on right-hand side is a prime attribute.

(3) If all given FDs satisfy this condition, then R is in 3NF.

Remark. Unfortunately, there is no shortcut, since we need keys to determine prime attributes.

Example. Consider $R(A, B, C, D)$ with $AB \to C$, $C \to D$, $D \to A$.

(1) Notice B must be in any key.

$$B^+ = B, \quad AB^+ = ABCD, \quad BC^+ = ABCD, \quad BD^+ = ABCD.$$

Hence, the keys of R are AB, BC, BD.

(2) All attributes are prime!

(3) Hence, R is in 3NF.

11.3 Synthesis algorithm

Similar to BCNF, if a table is NOT in 3NF, then we decompose it into smaller tables that are in 3NF.

Unlike BCNF decomposition which perform binary splits, a 3NF decomposition has only *one* split, which divides the table into two or more parts.

The idea is to create one table for each FD, and we simplify FDs to avoid redundant/unnecessary tables.

Definition 11.3. Given a set F of FDs, the **minimal basis** M of F is a simplified version of F satisfying the following conditions:

(1) M is equivalent to F.

(2) Every FD in M is a non-trivial and decomposed FD.

(3) No FD in M is redundant (i.e., cannot be derived from other FDs in M).

(4) For each FD in M, none of the attributes on left-hand side is redundant.

That is, if we remove an attribute from left-hand side, then the resulting FD is a new FD that cannot be derived from the original set of FDs.

Algorithm 11.4 (Minimal basis)**.** *Given a set F of FDs.*

(1) Simplify RHS: Decompose FDs so that each right-hand side contains only one attribute.

(2) Simplify LHS: Remove redundant attributes on left-hand side of each FD.

(3) Remove redundant FDs.

Example. Consider $F = \{A \to BD, AB \to C, C \to D, BC \to D\}$.

(1) After decomposing FDs, $F = \{A \to B, A \to D, AB \to C, C \to D, BC \to D\}$.

(2) Consider FDs with more than one attribute on LHS, i.e., $AB \to C$ and $BC \to D$.

For $AB \to C$, we check if $A \to C$ or $B \to C$ is implied by F. We compute closure: Since $A^+ = ABCD$, $A \to C$ holds. Since $B^+ = B$, $B \to C$ does not hold. Thus, B is redundant, so we can simplify $AB \to C$ to $A \to C$.

Note: More generally, if $ABC \to D$, we need to check three possibilities: $AB \to D$, $AC \to D$, $BC \to D$. Check if any holds using closure. If two hold, choose either one.

Hence, $F = \{A \to B, A \to D, A \to C, C \to D\}$.

(3) Notice $A \to D$ is implied by the other FDs (by computing A^+). Thus, $A \to D$ is redundant and should be removed.

Hence, $F = \{A \to B, A \to C, C \to D\}$.

Therefore, the minimal basis is $\{A \to B, A \to C, C \to D\}$.

Definition 11.5. A **canonical cover** is obtained from a minimal basis by combining FDs whose left-hand sides are the same.

The following is the algorithm to decompose a relation into 3NF.

Algorithm 11.6 (Synthesis algorithm)**.** *Suppose that R is not in BCNF.*

(1) Find a minimal basis of F.

(2) Calculate the canonical cover.

(3) For each FD, create a table that contains all attributes in the FD.

(4) Check if any *of the tables contain a key for R; if not, then create a table that contains a key for R.*

(5) Remove subsumed tables.

(4) - add new table containing a key, so that we can join the decomposed tables to recover the original table - to ensure lossless join decomposition

Example. Consider a relation $R(A, B, C, D, E, F)$ such that R is not in 3NF. What is the largest number of candidate keys in R?

Solution. For 3NF to be violated, at least one attribute, say F, is not prime, so there are at most 5 prime attributes A, B, C, D, E.

Thus, all candidate keys must be subsets of $\{A, B, C, D, E\}$.

If single attributes were keys, then only 5 candidate keys exist. If triples or larger sets were needed, the number of possible keys would shrink because larger sets overlap and are not minimal. The maximum number comes when pairs of attributes are sufficient to form keys.

From 5 attributes, the number of distinct 2-element subsets is $\binom{5}{2} = 10$. So if the functional dependencies are arranged so that every pair of primes determines the whole relation, then each of those 10 pairs is a candidate key. □

A MoE Schema

```
01 | CREATE TABLE IF NOT EXISTS regions (
02 |   name VARCHAR(16) PRIMARY KEY,
03 |   area VARCHAR(7) NOT NULL
04 | );
05 |
06 | CREATE TABLE IF NOT EXISTS addresses (
07 |   postal CHAR(6) PRIMARY KEY,
08 |   street VARCHAR(65) NOT NULL,
09 |   region VARCHAR(16) NOT NULL REFERENCES regions(name)
10 | );
11 |
12 | CREATE TABLE IF NOT EXISTS schools (
13 |   name VARCHAR(64) PRIMARY KEY,
14 |   postal CHAR(6) NOT NULL REFERENCES addresses(postal),
15 |   phone INT NOT NULL
16 |     CHECK (phone BETWEEN 60000000 AND 69999999),
17 |   email VARCHAR(64) NOT NULL,
18 |   sch_type VARCHAR(32) NOT NULL
19 |     CHECK (sch_type IN ('Government', 'Government-Aided', 'Independent', '
       Specialised')),
20 |   sch_nature VARCHAR(5) NOT NULL
21 |     CHECK (sch_nature IN ('co-ed', 'boys', 'girls')),
22 |   sch_session VARCHAR(16) NOT NULL,
23 |   sch_level VARCHAR(32) NOT NULL
24 | );
25 |
26 | CREATE TABLE IF NOT EXISTS stations (
27 |   code VARCHAR(5) PRIMARY KEY,
28 |   line VARCHAR(32) NOT NULL,
29 |   name VARCHAR(32) NOT NULL
30 | );
31 |
32 | CREATE TABLE IF NOT EXISTS nearby (
33 |   school VARCHAR(64) REFERENCES schools(name),
34 |   station VARCHAR(5) REFERENCES stations(code),
35 |   PRIMARY KEY (school, station)
36 | );
37 |
38 | CREATE TABLE IF NOT EXISTS programmes (
39 |   school VARCHAR(64) REFERENCES schools(name),
40 |   program VARCHAR(64),
41 |   PRIMARY KEY (school, program)
42 | );
43 |
44 | CREATE TABLE IF NOT EXISTS languages (
45 |   name VARCHAR(64) PRIMARY KEY
46 | );
47 |
48 | CREATE TABLE IF NOT EXISTS mothertongues (
49 |   school VARCHAR(64) REFERENCES schools(name),
50 |   lang VARCHAR(64) REFERENCES languages(name),
51 |   PRIMARY KEY (school, lang)
52 | );
53 |
54 | CREATE TABLE IF NOT EXISTS activities (
```

```
55 |     name VARCHAR(64) PRIMARY KEY,
56 |     category VARCHAR(64) NOT NULL
57 |   );
58 |
59 |   CREATE TABLE IF NOT EXISTS ccas (
60 |     school VARCHAR(64) REFERENCES schools(name),
61 |     activity VARCHAR(64) REFERENCES activities(name),
62 |     PRIMARY KEY (school, activity)
63 |   );
64 |
65 |   CREATE TABLE IF NOT EXISTS studies (
66 |     school VARCHAR(64) REFERENCES schools(name),
67 |     name VARCHAR(64),
68 |     PRIMARY KEY (school, name)
69 |   );
70 |
71 |   CREATE TABLE IF NOT EXISTS subjects (
72 |     school VARCHAR(64) REFERENCES schools(name),
73 |     name VARCHAR(64),
74 |     PRIMARY KEY (school, name)
75 |   );
```